



Lab Task Report

Only for course Teacher					
	Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage	25%	50%	75%	100%	
Clarity					
Content Quality					
Spelling & Grammar					
Organized on and Form a ng					
Total obtained mark					
Comments					

Semester: January-June 2026

Student Name: Protul Devnath

Student ID: 0022520005101o89

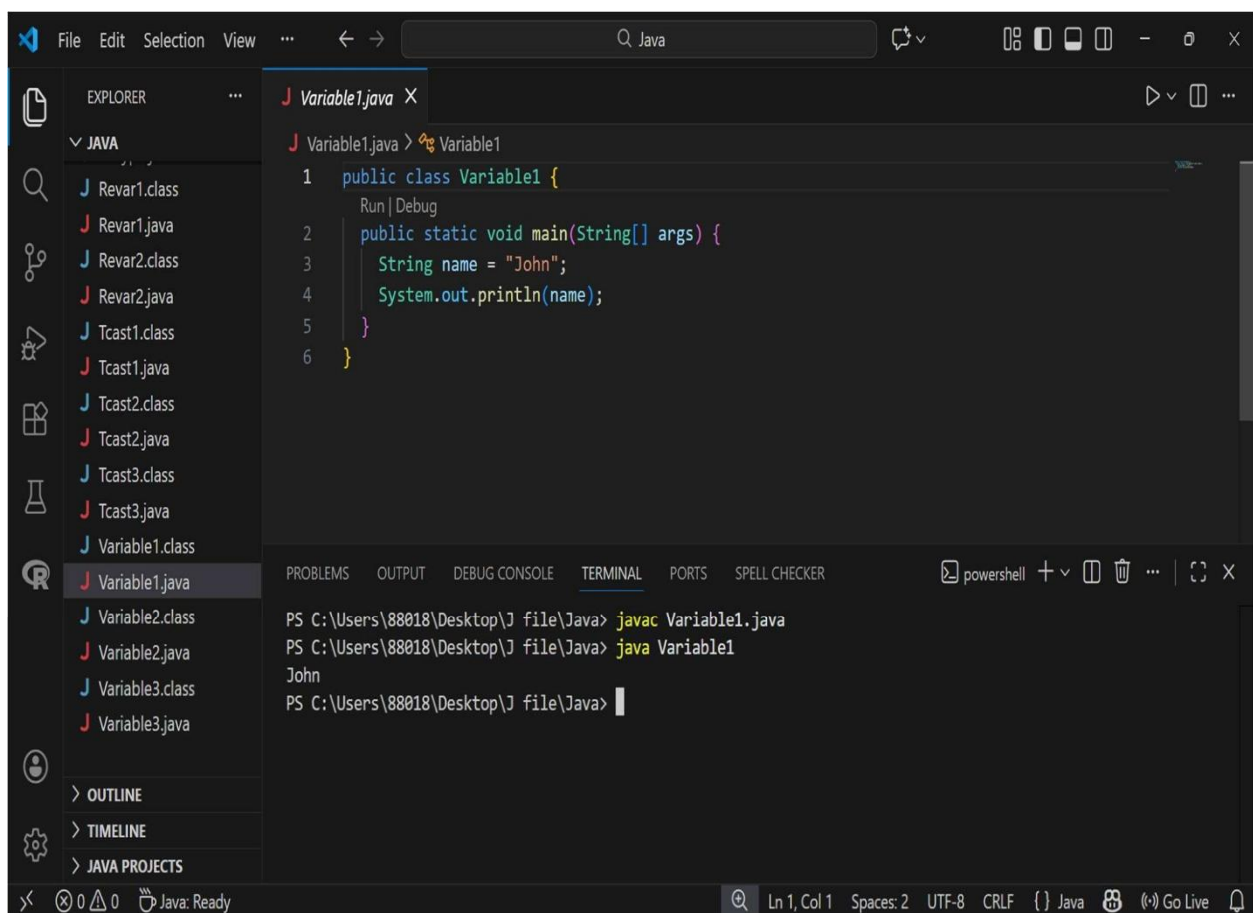
Batch: 45th

Sec on: C

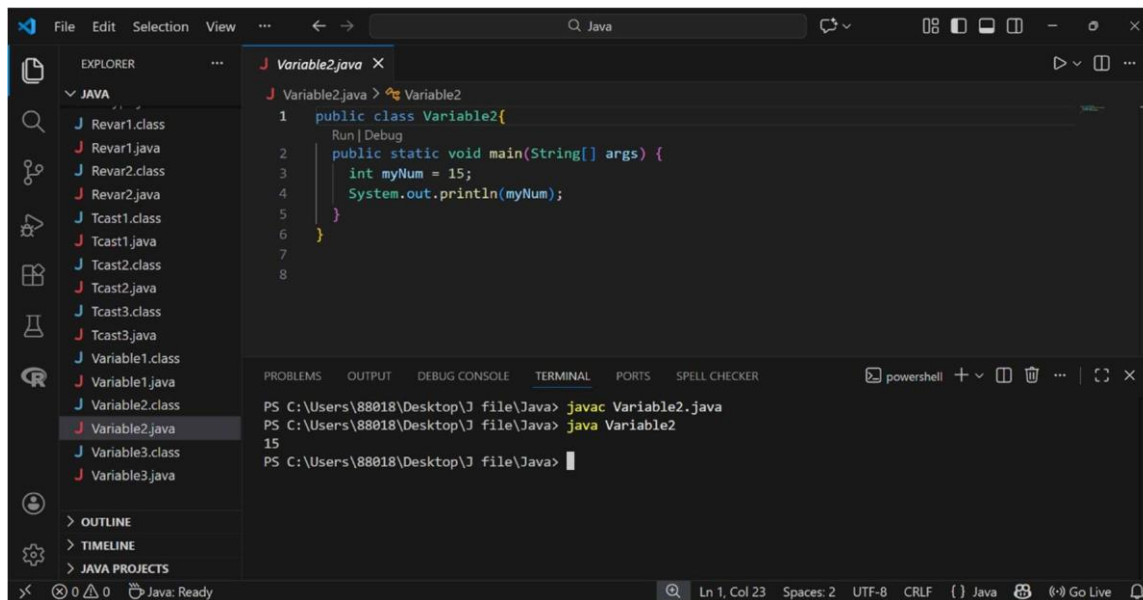
Course Code: 124

Course Name: Object Oriented Programming

Course Teacher Name: Debabarata Mallick



2. This Java program illustrates the declaration and initialization of an integer variable, assigning the numerical value of 15 to the identifier `myNum`. The code subsequently invokes the `System.out.println()` function to print the stored integer value to the terminal, as confirmed by the program output in the terminal window.



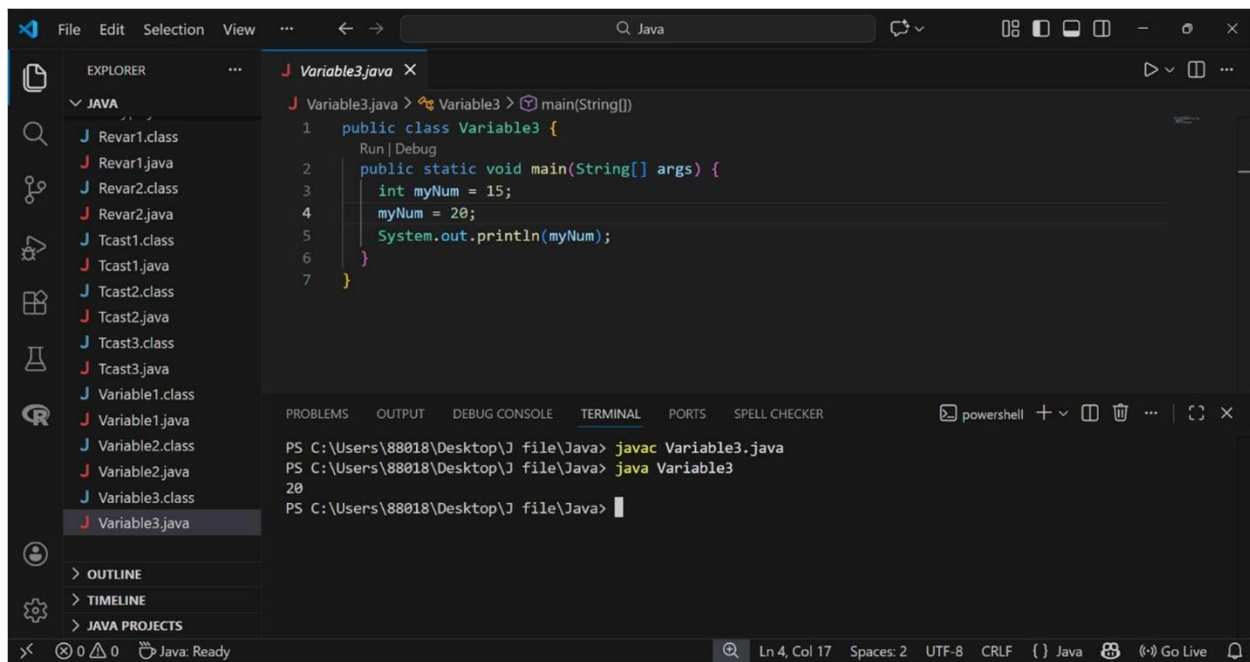
The screenshot shows an IDE with a file explorer on the left containing various Java files. The main editor displays the code for `Variable2.java`:

```
1 public class Variable2 {  
2     public static void main(String[] args) {  
3         int myNum = 15;  
4         System.out.println(myNum);  
5     }  
6 }  
7  
8
```

The bottom panel shows the terminal output:

```
PS C:\Users\88018\Desktop\J file\Java> javac Variable2.java  
PS C:\Users\88018\Desktop\J file\Java> java Variable2  
15  
PS C:\Users\88018\Desktop\J file\Java>
```

3. This Java program demonstrates variable reassignment, where an integer variable `myNum` is initially declared with the value 15 and subsequently updated to 20. The final value is then printed to the console using `System.out.println()`, illustrating that variables can hold different values over time during program execution.

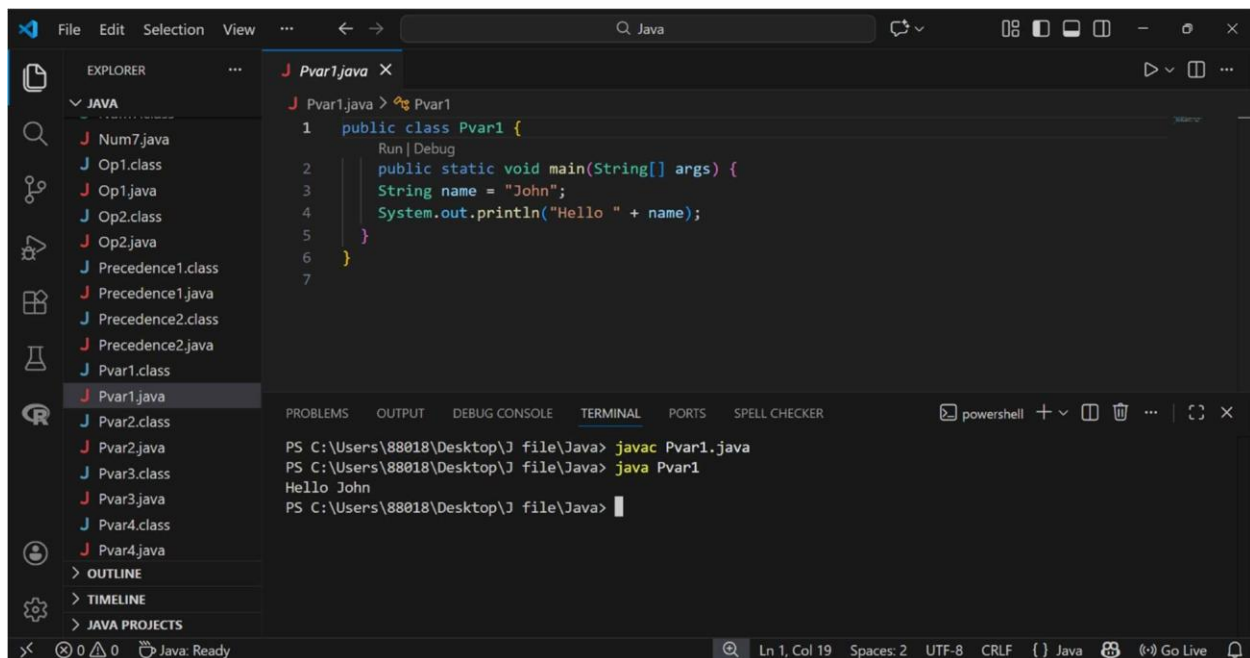


```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Revar1.class
    Revar1.java
    Revar2.class
    Revar2.java
    Tcast1.class
    Tcast1.java
    Tcast2.class
    Tcast2.java
    Tcast3.class
    Tcast3.java
    Variable1.class
    Variable1.java
    Variable2.class
    Variable2.java
    Variable3.class
    Variable3.java
  > OUTLINE
  > TIMELINE
  > JAVA PROJECTS

J Variable3.java
1 public class Variable3 {
2     public static void main(String[] args) {
3         int myNum = 15;
4         myNum = 20;
5         System.out.println(myNum);
6     }
7 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Variable3.java
PS C:\Users\88018\Desktop\J file\Java> java Variable3
20
PS C:\Users\88018\Desktop\J file\Java>
```

4. This program highlights string concatenation by using the + operator to combine the string literal "Hello " with the value stored in the variable name. The resulting combined string, "Hello John", is then printed to the terminal, showing how static text can be dynamically joined with variable data.



```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Num7.java
    Op1.class
    Op1.java
    Op2.class
    Op2.java
    Precedence1.class
    Precedence1.java
    Precedence2.class
    Precedence2.java
    Pvar1.class
    Pvar1.java
    Pvar2.class
    Pvar2.java
    Pvar3.class
    Pvar3.java
    Pvar4.class
    Pvar4.java
  > OUTLINE
  > TIMELINE
  > JAVA PROJECTS

J Pvar1.java
1 public class Pvar1 {
2     public static void main(String[] args) {
3         String name = "John";
4         System.out.println("Hello " + name);
5     }
6 }
7

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Pvar1.java
PS C:\Users\88018\Desktop\J file\Java> java Pvar1
Hello John
PS C:\Users\88018\Desktop\J file\Java>
```

5. This program demonstrates complex string manipulation by combining two separate string variables, firstName and lastName, into a new variable called fullName for display.

The screenshot shows an IDE with a file explorer on the left containing various Java files. The main editor displays the code for `Pvar2.java`, which defines a `main` method that concatenates the strings "John " and "Doe" into `fullName` and prints it. The terminal at the bottom shows the command `javac Pvar2.java` being executed, followed by `java Pvar2`, which results in the output `John Doe`.

```
public class Pvar2 {  
    public static void main(String[] args) {  
        String firstName = "John ";  
        String lastName = "Doe";  
        String fullName = firstName + lastName;  
        System.out.println(fullName);  
    }  
}
```

```
PS C:\Users\88018\Desktop\J file\Java> javac Pvar2.java  
PS C:\Users\88018\Desktop\J file\Java> java Pvar2  
John Doe  
PS C:\Users\88018\Desktop\J file\Java>
```

6. This code performs arithmetic addition by using the + operator to calculate and print the sum of two integer variables, x and y.

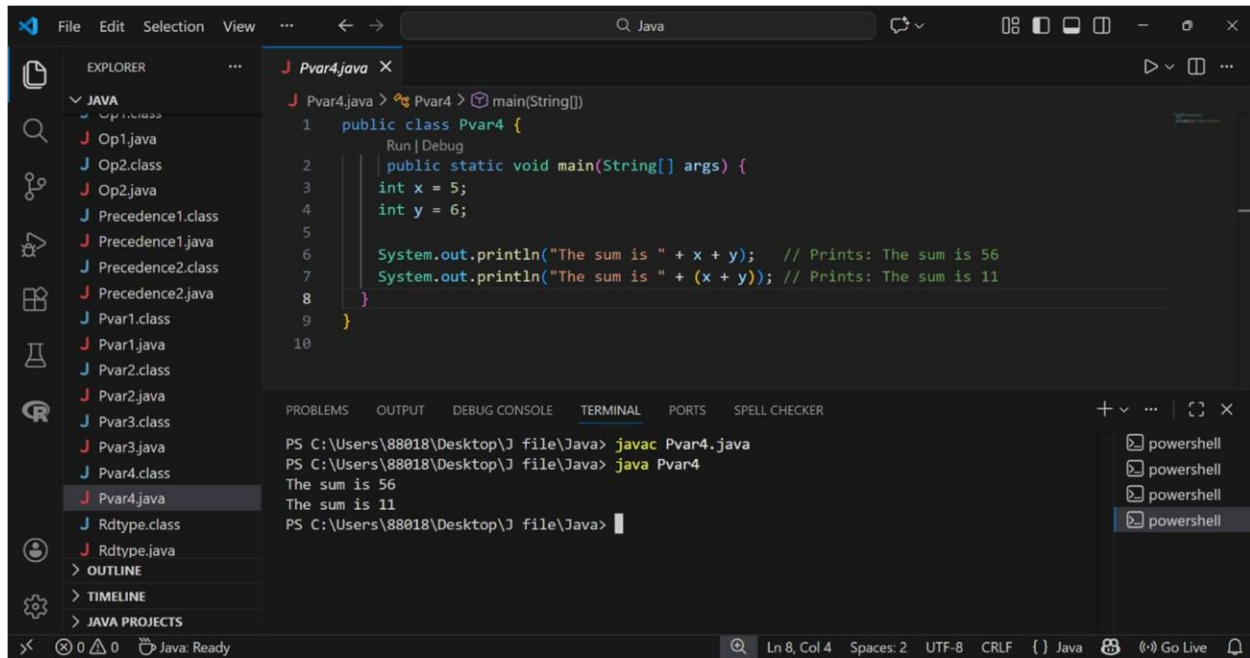
The screenshot shows the same IDE with the file explorer. The main editor displays the code for `Pvar3.java`, which defines a `main` method that initializes two integer variables, `x` and `y`, with values 5 and 6 respectively, and prints their sum using `System.out.println(x + y)`. The terminal shows the command `javac Pvar3.java` being executed, followed by `java Pvar3`, which results in the output `11`.

```
public class Pvar3 {  
    public static void main(String[] args) {  
        int x = 5;  
        int y = 6;  
        System.out.println(x + y); // Print the value of x + y  
    }  
}
```

```
PS C:\Users\88018\Desktop\J file\Java> javac Pvar3.java  
PS C:\Users\88018\Desktop\J file\Java> java Pvar3  
11  
PS C:\Users\88018\Desktop\J file\Java>
```

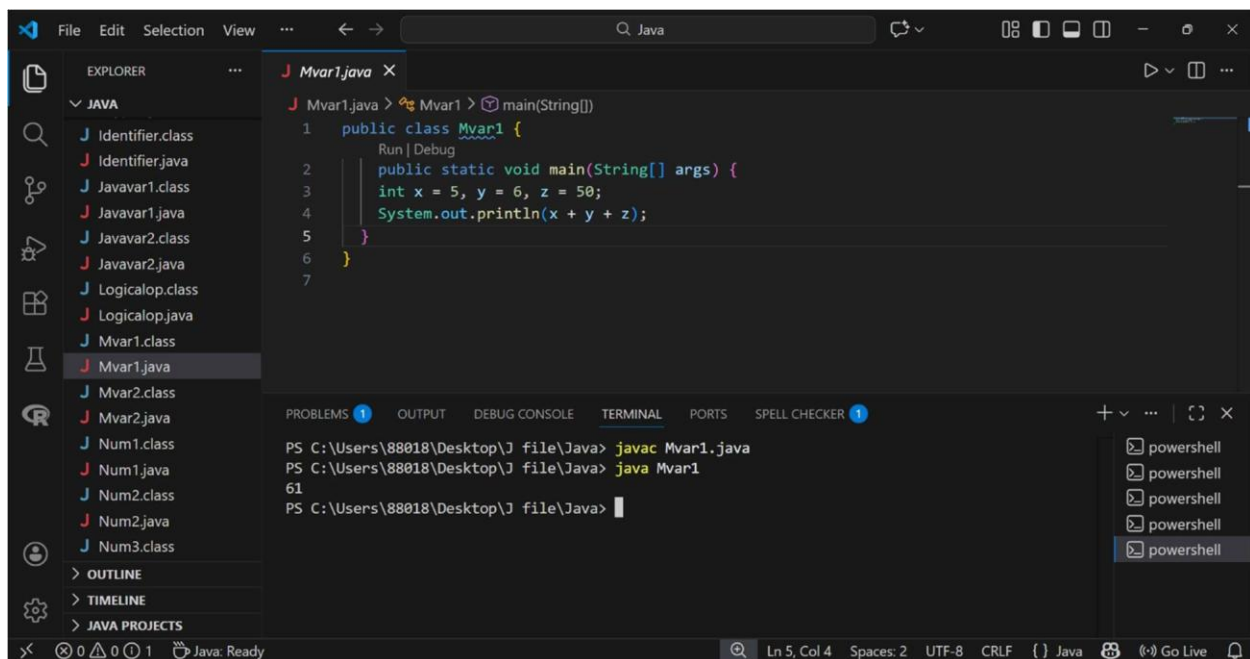
7. This program highlights the impact of operator precedence by showing how parentheses can force mathematical addition (result 11) rather than default string concatenation.

(result 56).



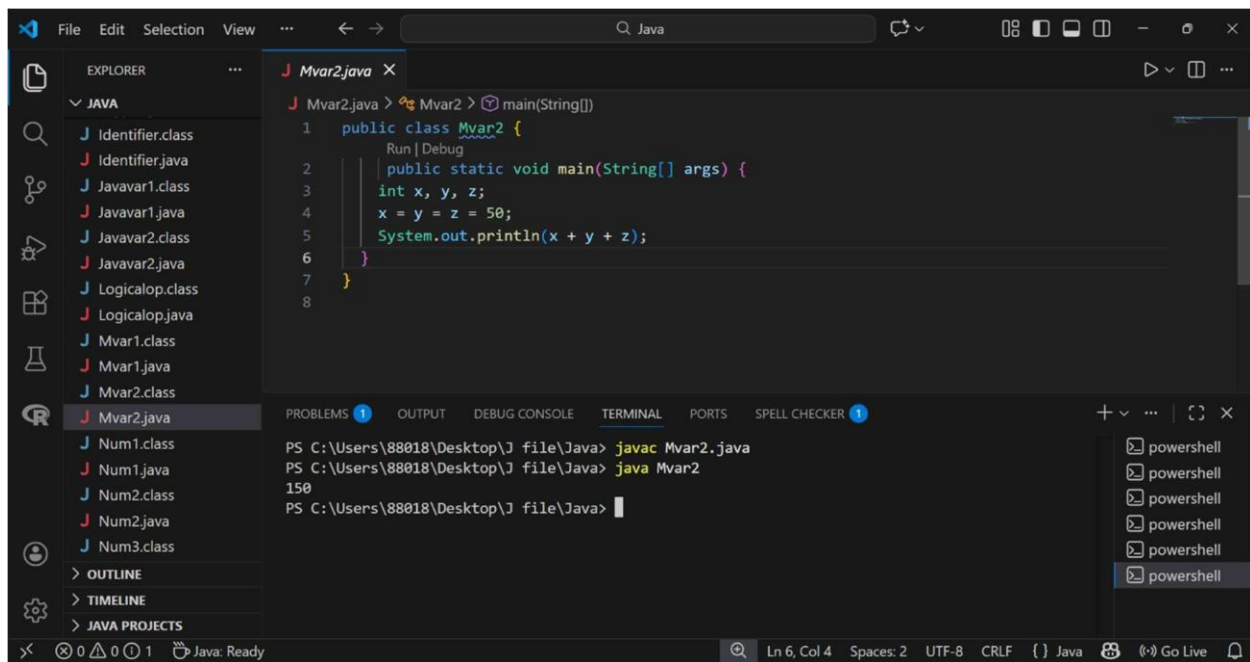
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Op1.java
    Op2.class
    Op2.java
    Precedence1.class
    Precedence1.java
    Precedence2.class
    Precedence2.java
    Pvar1.class
    Pvar1.java
    Pvar2.class
    Pvar2.java
    Pvar3.class
    Pvar3.java
    Pvar4.class
    Pvar4.java
    Rdtype.class
    Rdtype.java
  > OUTLINE
  > TIMELINE
  > JAVA PROJECTS
Pvar4.java
1 public class Pvar4 {
2     public static void main(String[] args) {
3         int x = 5;
4         int y = 6;
5
6         System.out.println("The sum is " + x + y); // Prints: The sum is 56
7         System.out.println("The sum is " + (x + y)); // Prints: The sum is 11
8     }
9
10
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Pvar4.java
PS C:\Users\88018\Desktop\J file\Java> java Pvar4
The sum is 56
The sum is 11
PS C:\Users\88018\Desktop\J file\Java>
powershell
powershell
powershell
powershell
Ln 8, Col 4 Spaces: 2 UTF-8 CRLF () Java Go Live
```

8. This code demonstrates multiple variable declaration on a single line, initializing three separate integers (x, y, and z) and printing their collective sum.



```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Identifier.class
    Identifier.java
    Javavar1.class
    Javavar1.java
    Javavar2.class
    Javavar2.java
    Logicalop.class
    Logicalop.java
    Mvar1.class
    Mvar1.java
    Mvar2.class
    Mvar2.java
    Num1.class
    Num1.java
    Num2.class
    Num2.java
    Num3.class
  > OUTLINE
  > TIMELINE
  > JAVA PROJECTS
Mvar1.java
1 public class Mvar1 {
2     public static void main(String[] args) {
3         int x = 5, y = 6, z = 50;
4         System.out.println(x + y + z);
5     }
6
7
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL 1 PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Mvar1.java
PS C:\Users\88018\Desktop\J file\Java> java Mvar1
61
PS C:\Users\88018\Desktop\J file\Java>
powershell
powershell
powershell
powershell
powershell
Ln 5, Col 4 Spaces: 2 UTF-8 CRLF () Java Go Live
```

9. This script illustrates multiple variable assignment to a common value, where three previously declared integers are all set to 50 in a single statement.



```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Identifier.class
    Identifier.java
    Javavar1.class
    Javavar1.java
    Javavar2.class
    Javavar2.java
    Logicalop.class
    Logicalop.java
    Mvar1.class
    Mvar1.java
    Mvar2.class
    Mvar2.java
    Num1.class
    Num1.java
    Num2.class
    Num2.java
    Num3.class
  OUTLINE
  TIMELINE
  JAVA PROJECTS

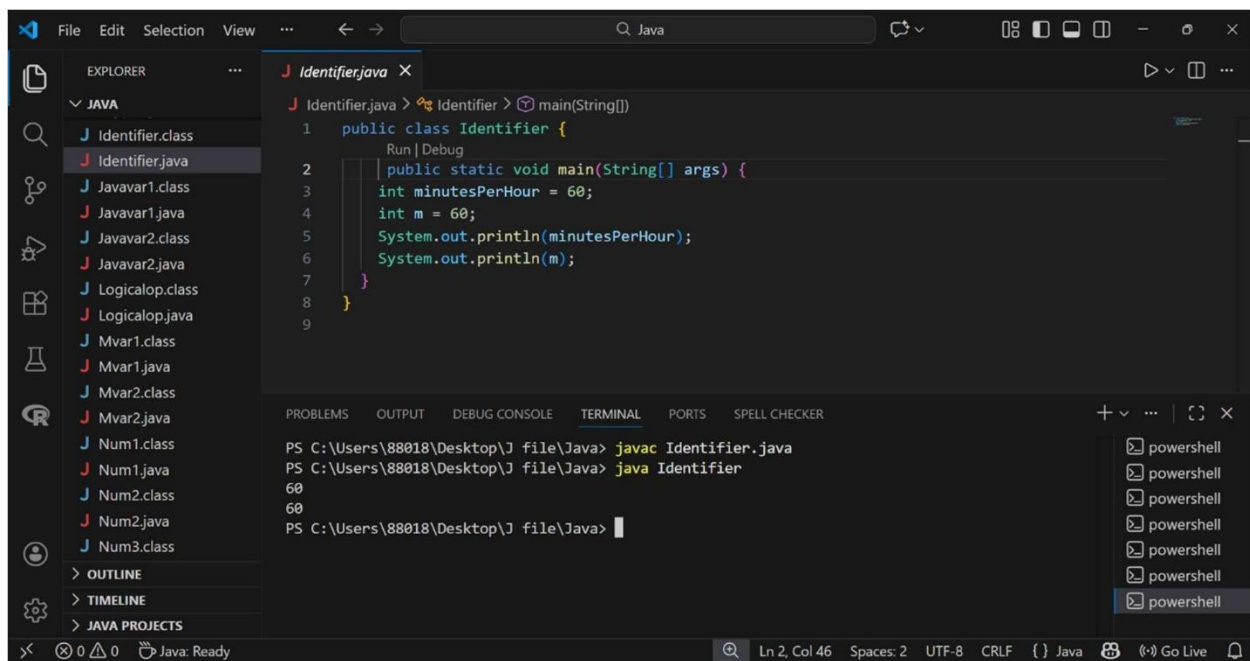
Mvar2.java
1 public class Mvar2 {
  Run | Debug
  public static void main(String[] args) {
2     int x, y, z;
3     x = y = z = 50;
4     System.out.println(x + y + z);
5 }
6
7
8

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Mvar2.java
PS C:\Users\88018\Desktop\J file\Java> java Mvar2
150
PS C:\Users\88018\Desktop\J file\Java>

powershell
powershell
powershell
powershell
powershell

Ln 6, Col 4 Spaces: 2 UTF-8 CRLF () Java Go Live
```

10. This program illustrates the importance of meaningful identifiers, comparing the descriptive `minutesPerHour` against the vague variable name `m`.



```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Identifier.class
    Identifier.java
    Javavar1.class
    Javavar1.java
    Javavar2.class
    Javavar2.java
    Logicalop.class
    Logicalop.java
    Mvar1.class
    Mvar1.java
    Mvar2.class
    Mvar2.java
    Num1.class
    Num1.java
    Num2.class
    Num2.java
    Num3.class
  OUTLINE
  TIMELINE
  JAVA PROJECTS

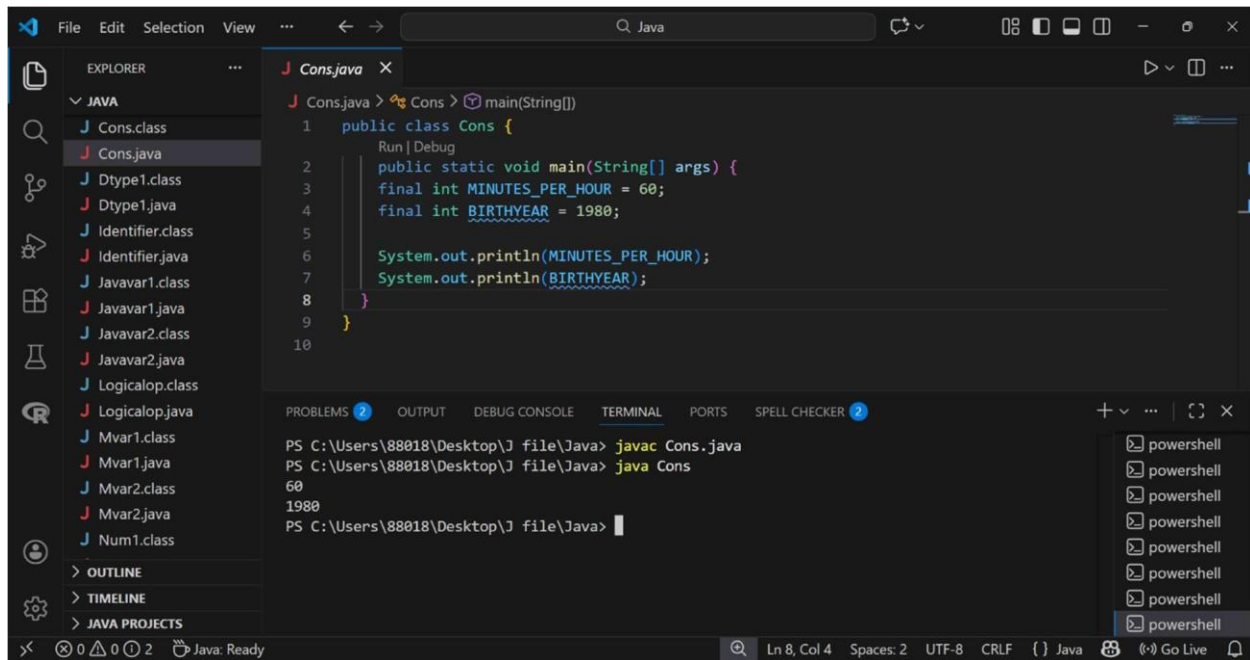
Identifier.java
1 public class Identifier {
  Run | Debug
  public static void main(String[] args) {
2     int minutesPerHour = 60;
3     int m = 60;
4     System.out.println(minutesPerHour);
5     System.out.println(m);
6 }
7
8
9

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Identifier.java
PS C:\Users\88018\Desktop\J file\Java> java Identifier
60
60
PS C:\Users\88018\Desktop\J file\Java>

powershell
powershell
powershell
powershell
powershell
powershell

Ln 2, Col 46 Spaces: 2 UTF-8 CRLF () Java Go Live
```

11. This script demonstrates the use of the final keyword to declare constants, ensuring that the values of MINUTES_PER_HOUR and BIRTHYEAR cannot be modified during execu on.



The screenshot shows an IDE with a Java project. The Explorer panel on the left lists several files, including `Cons.class`, `Cons.java`, `Dtype1.class`, `Dtype1.java`, `Identifier.class`, `Identifier.java`, `Javavar1.class`, `Javavar1.java`, `Javavar2.class`, `Javavar2.java`, `Logicalop.class`, `Logicalop.java`, `Mvar1.class`, `Mvar1.java`, `Mvar2.class`, `Mvar2.java`, and `Num1.class`. The main editor displays the source code for `Cons.java`, which is a public class named `Cons` with a `main` method. The code uses the `final` keyword to declare two constants: `MINUTES_PER_HOUR` and `BIRTHYEAR`. The `main` method prints the values of these constants. The terminal at the bottom shows the command `javac Cons.java` being executed, followed by the output `60` and `1980`. The status bar at the bottom indicates the current line and column (Ln 8, Col 4) and the file encoding (UTF-8).

```
1 public class Cons {  
2     Run | Debug  
3     public static void main(String[] args) {  
4         final int MINUTES_PER_HOUR = 60;  
5         final int BIRTHYEAR = 1980;  
6  
7         System.out.println(MINUTES_PER_HOUR);  
8         System.out.println(BIRTHYEAR);  
9     }  
10 }
```

PS C:\Users\88018\Desktop\J file\Java> javac Cons.java
PS C:\Users\88018\Desktop\J file\Java> java Cons
60
1980
PS C:\Users\88018\Desktop\J file\Java>

12. This program provides a comprehensive example of diverse data type usage, declaring and displaying String, int, float, and char variables in a single report-style output.

The screenshot shows an IDE with the Explorer panel on the left listing Java files. The main editor displays `Revar1.java` with the following code:

```
1 public class Revar1 {  
2     public static void main(String[] args) {  
3         String studentName = "John Doe";  
4         int studentID = 15;  
5         int studentAge = 23;  
6         float studentFee = 75.25f;  
7         char studentGrade = 'B';  
8         System.out.println("Student name: " + studentName);  
9         System.out.println("Student id: " + studentID);  
10        System.out.println("Student age: " + studentAge);  
11        System.out.println("Student fee: " + studentFee);  
12        System.out.println("Student grade: " + studentGrade);  
13    }  
14 }
```

The bottom panel shows the terminal output:

```
PS C:\Users\88018\Desktop\J file\Java> java Revar1  
Student name: John Doe  
Student id: 15  
Student age: 23  
Student fee: 75.25  
Student grade: B  
PS C:\Users\88018\Desktop\J file\Java>
```

13. This code illustrates a practical application of variables by using integers to calculate and display the area of a rectangle through a mathematical formula.

The screenshot shows an IDE with the Explorer panel on the left listing Java files. The main editor displays `Revar2.java` with the following code:

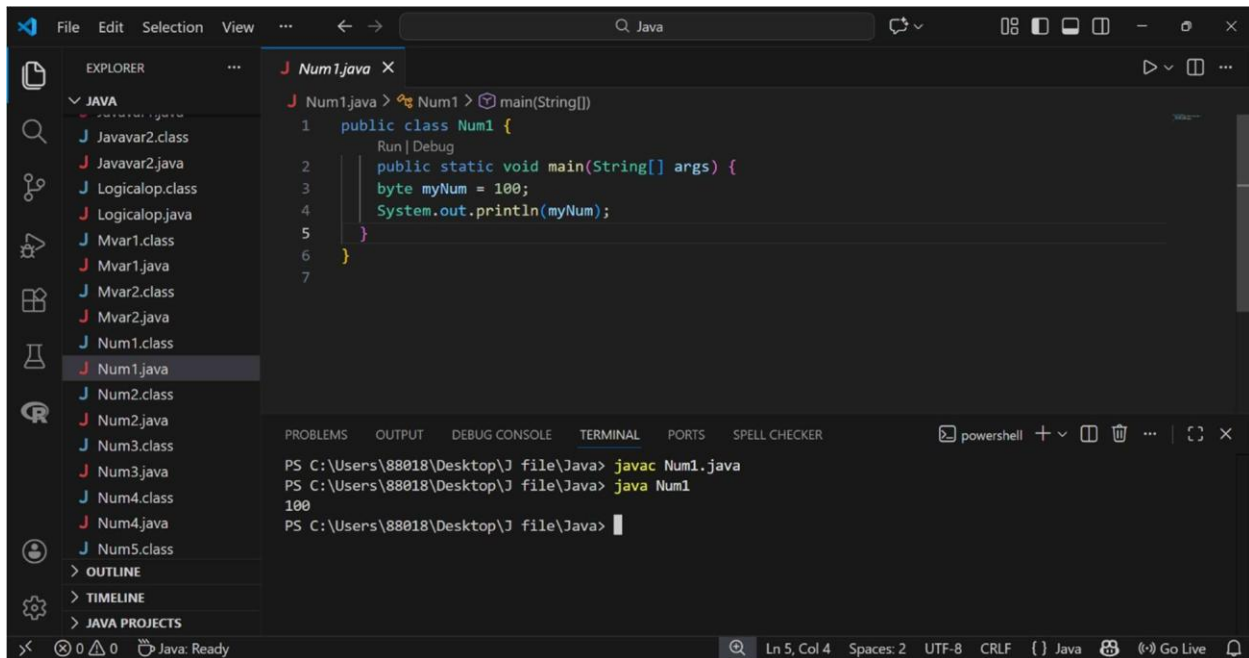
```
1 public class Revar2 {  
2     public static void main(String[] args) {  
3         int length = 4;  
4         int width = 6;  
5         int area;  
6         area = length * width;  
7         System.out.println("Length is: " + length);  
8         System.out.println("Width is: " + width);  
9         System.out.println("Area of the rectangle is: " + area);  
10    }  
11 }  
12 }
```

The bottom panel shows the terminal output:

```
PS C:\Users\88018\Desktop\J file\Java> javac Revar2.java  
PS C:\Users\88018\Desktop\J file\Java> java Revar2  
Length is: 4  
Width is: 6  
Area of the rectangle is: 24  
PS C:\Users\88018\Desktop\J file\Java>
```

Data Types sec on:

1. The provided Java code demonstrates the declaration of a byte data type variable named myNum, assigns it a value of 100, and then outputs that value to the console using the System.out.println() method.

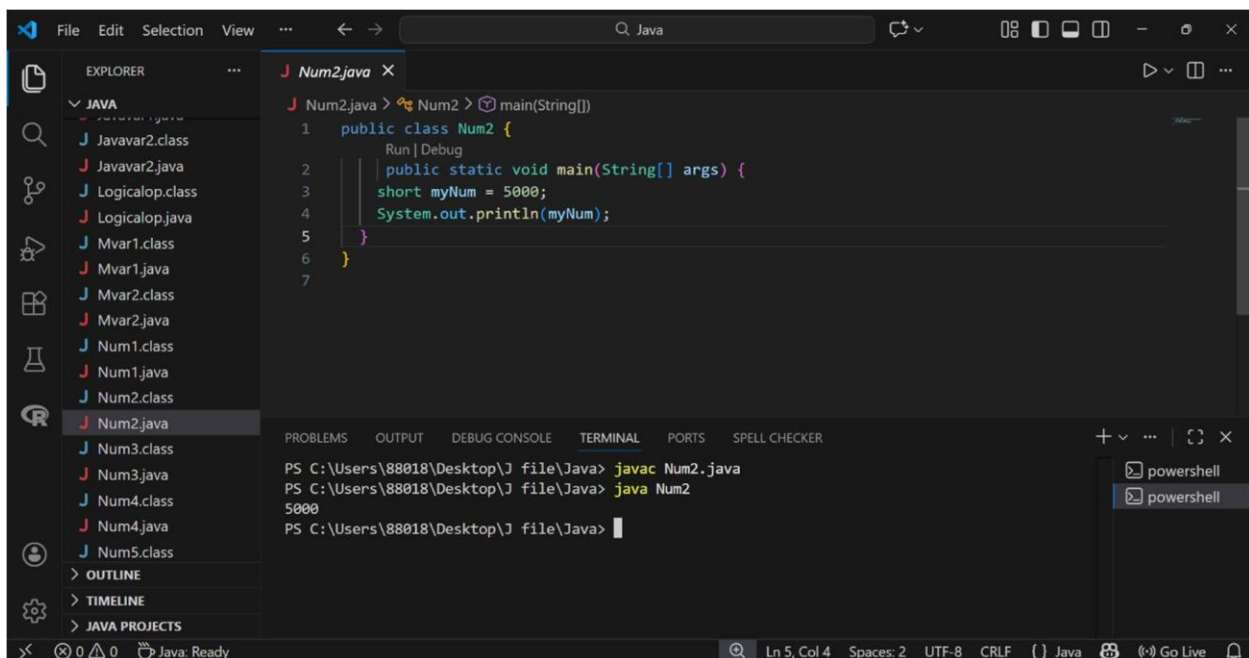


The screenshot shows an IDE with the Explorer panel on the left listing several Java files. The main editor displays the code for Num1.java. The code defines a public class Num1 with a main method. Inside the main method, a byte variable myNum is declared and assigned the value 100, which is then printed to the console using System.out.println(myNum). The terminal at the bottom shows the command prompt where the program is compiled and run, resulting in the output 100.

```
1 public class Num1 {  
2     public static void main(String[] args) {  
3         byte myNum = 100;  
4         System.out.println(myNum);  
5     }  
6 }  
7
```

```
PS C:\Users\88018\Desktop\J file\Java> javac Num1.java  
PS C:\Users\88018\Desktop\J file\Java> java Num1  
100  
PS C:\Users\88018\Desktop\J file\Java>
```

2. This Java program illustrates the use of the short primitive data type to store a 16-bit integer value of 5000 in the variable myNum before printing it to the terminal.

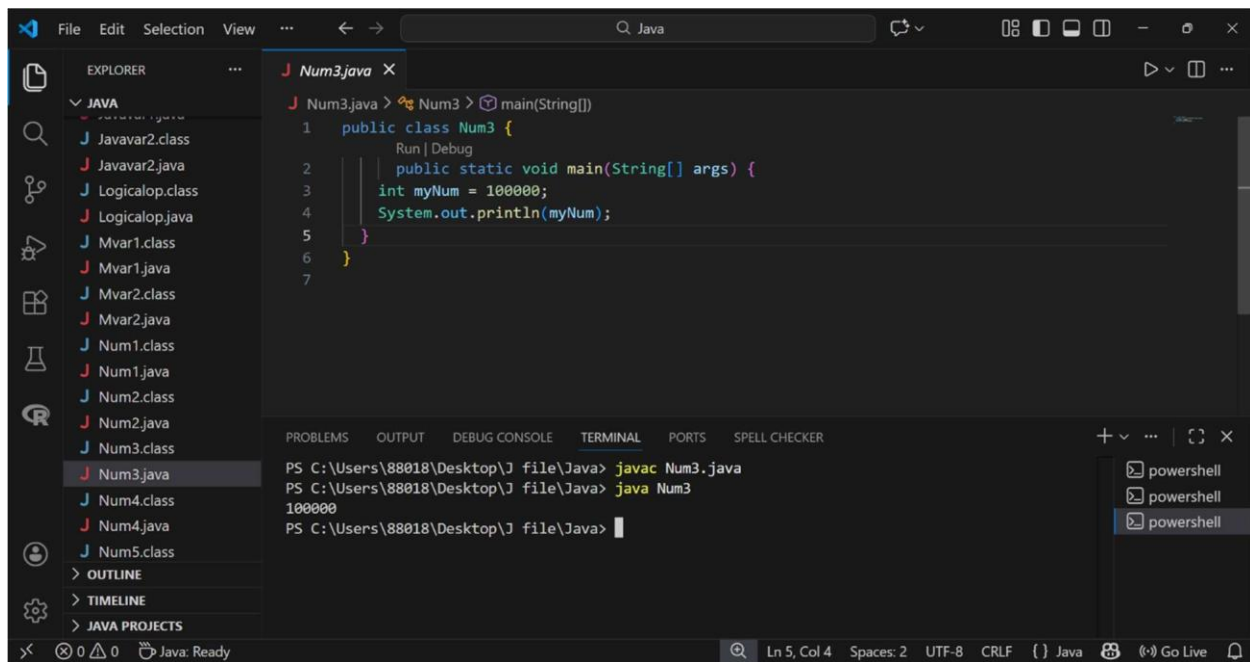


The screenshot shows an IDE with the Explorer panel on the left listing several Java files. The main editor displays the code for Num2.java. The code defines a public class Num2 with a main method. Inside the main method, a short variable myNum is declared and assigned the value 5000, which is then printed to the console using System.out.println(myNum). The terminal at the bottom shows the command prompt where the program is compiled and run, resulting in the output 5000.

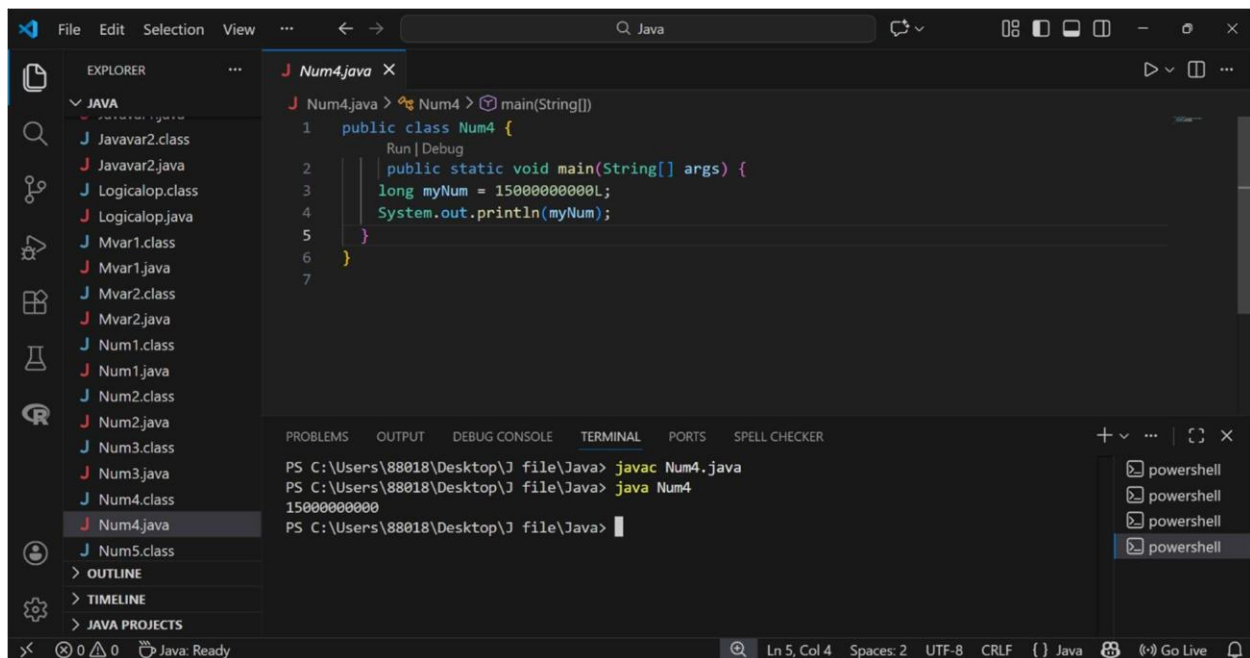
```
1 public class Num2 {  
2     public static void main(String[] args) {  
3         short myNum = 5000;  
4         System.out.println(myNum);  
5     }  
6 }  
7
```

```
PS C:\Users\88018\Desktop\J file\Java> javac Num2.java  
PS C:\Users\88018\Desktop\J file\Java> java Num2  
5000  
PS C:\Users\88018\Desktop\J file\Java>
```

3. This Java program demonstrates the use of the int data type to store a large 32-bit integer value of 100000 in the variable myNum and displays it in the terminal.



4. This Java program utilizes the long data type to store an exceptionally large 64-bit integer value, indicated by the L suffix, and prints it to the console.



5. This program demonstrates the float data type for storing single-precision decimal numbers, identified by the required f suffix.

The screenshot shows an IDE with a project named 'J file'. The Explorer pane on the left lists several Java files, including 'Num5.java'. The main editor displays the code for 'Num5.java':

```
1 public class Num5 {  
2     public static void main(String[] args) {  
3         float myNum = 5.75f;  
4         System.out.println(myNum);  
5     }  
6 }  
7
```

The bottom panel shows the 'TERMINAL' tab with the following commands and output:

```
PS C:\Users\88018\Desktop\J file\Java> javac Num5.java  
PS C:\Users\88018\Desktop\J file\Java> java Num5  
5.75  
PS C:\Users\88018\Desktop\J file\Java>
```

The status bar at the bottom indicates 'Ln 5, Col 4', 'Spaces: 2', 'UTF-8', 'CRLF', and 'Java'.

6. This code utilizes the double data type to store a double-precision floating-point number, offering higher precision for decimal values.

The screenshot shows the same IDE with the project 'J file'. The Explorer pane lists files, including 'Num6.java'. The main editor displays the code for 'Num6.java':

```
1 public class Num6 {  
2     public static void main(String[] args) {  
3         double myNum = 19.99d;  
4         System.out.println(myNum);  
5     }  
6 }  
7
```

The bottom panel shows the 'TERMINAL' tab with the following commands and output:

```
PS C:\Users\88018\Desktop\J file\Java> javac Num6.java  
PS C:\Users\88018\Desktop\J file\Java> java Num6  
19.99  
PS C:\Users\88018\Desktop\J file\Java>
```

The status bar at the bottom indicates 'Ln 5, Col 4', 'Spaces: 2', 'UTF-8', 'CRLF', and 'Java'.

7. Features the use of scientific notation (e.g., 35e3f) to assign powers-of-ten values to floating-point and double variables.

```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Mvar1.java
    Mvar2.class
    Mvar2.java
    Num1.class
    Num1.java
    Num2.class
    Num2.java
    Num3.class
    Num3.java
    Num4.class
    Num4.java
    Num5.class
    Num5.java
    Num6.class
    Num6.java
    Num7.class
    Num7.java
    On1.class
  > OUTLINE
  > TIMELINE
  > JAVA PROJECTS

J Num7.java X
J Num7.java > Num7 > main(String[])
1 public class Num7 {
  Run | Debug
2   public static void main(String[] args) {
3     float f1 = 35e3f;
4     double d1 = 12E4d;
5     System.out.println(f1);
6     System.out.println(d1);
7   }
8 }
9

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Num7.java
PS C:\Users\88018\Desktop\J file\Java> java Num7
35000.0
120000.0
PS C:\Users\88018\Desktop\J file\Java>

Ln 7, Col 4 Spaces: 2 UTF-8 CRLF () Java Go Live
```

8. Introduces the boolean data type, which is used to store logical values represent only true or false states.

```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Arop4.java
    Arop5.class
    Arop5.java
    Asop.class
    Asop.java
    Bool1.class
    Bool1.java
    Char1.class
    Char1.java
    Char2.class
    Char2.java
    Char3.class
    Char3.java
    Comop1.class
    Comop1.java
    Comop2.class
    Comop2.java
  > OUTLINE
  > TIMELINE
  > JAVA PROJECTS

J Bool1.java X
J Bool1.java > Bool1 > main(String[])
1 public class Bool1 {
  Run | Debug
2   public static void main(String[] args) {
3     boolean isJavaFun = true;
4     boolean isFishTasty = false;
5     System.out.println(isJavaFun);
6     System.out.println(isFishTasty);
7   }
8 }
9

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Bool1.java
PS C:\Users\88018\Desktop\J file\Java> java Bool1
true
false
PS C:\Users\88018\Desktop\J file\Java>

Ln 7, Col 4 Spaces: 2 UTF-8 CRLF () Java Go Live
```

9. Demonstrates the char data type used to store a single character, enclosed in single quotes.

The screenshot shows an IDE with a file explorer on the left containing various Java files. The main editor displays `Char1.java` with the following code:

```
1 public class Char1 {  
2     Run | Debug  
3     public static void main(String[] args) {  
4         char myGrade = 'B';  
5         System.out.println(myGrade);  
6     }  
7 }
```

The bottom panel shows the terminal with the following commands and output:

```
PS C:\Users\88018\Desktop\J file\Java> javac Char1.java  
PS C:\Users\88018\Desktop\J file\Java> java Char1  
B  
PS C:\Users\88018\Desktop\J file\Java>
```

10. Illustrates how ASCII values can be assigned to char variables to display their corresponding characters.

The screenshot shows an IDE with a file explorer on the left. The main editor displays `Char2.java` with the following code:

```
1 public class Char2 {  
2     Run | Debug  
3     public static void main(String[] args) {  
4         char myVar1 = 65, myVar2 = 66, myVar3 = 67;  
5         System.out.println(myVar1);  
6         System.out.println(myVar2);  
7         System.out.println(myVar3);  
8     }  
9 }
```

The bottom panel shows the terminal with the following commands and output:

```
PS C:\Users\88018\Desktop\J file\Java> javac Char2.java  
PS C:\Users\88018\Desktop\J file\Java> java Char2  
A  
B  
C  
PS C:\Users\88018\Desktop\J file\Java>
```

11. Introduces the String reference data type, which is used to store and display sequences of characters or text.

```

J Char3.java > Char3 > main(String[])
1 public class Char3 {
    Run | Debug
2     public static void main(String[] args) {
3         String greeting = "Hello World";
4         System.out.println(greeting);
5     }
6 }
7

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

```

PS C:\Users\88018\Desktop\J file\Java> javac Char3.java
PS C:\Users\88018\Desktop\J file\Java> java Char3
Hello World
PS C:\Users\88018\Desktop\J file\Java>

```

Ln 5, Col 4 Spaces: 2 UTF-8 CRLF () Java Go Live

12. Demonstrates a practical application by combining multiple data types (int, float, char) to calculate and format a total cost output.

```

J Rdtype.java > Rdtype > main(String[])
1 public class Rdtype {
    Run | Debug
2     public static void main(String[] args) {
3         int items = 50;
4         float costPerItem = 9.99f;
5         float totalCost = items * costPerItem;
6         char currency = '$';
7         System.out.println("Number of items: " + items);
8         System.out.println("Cost per item: " + costPerItem + currency);
9         System.out.println("Total cost = " + totalCost + currency);
10    }
11 }
12

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1

```

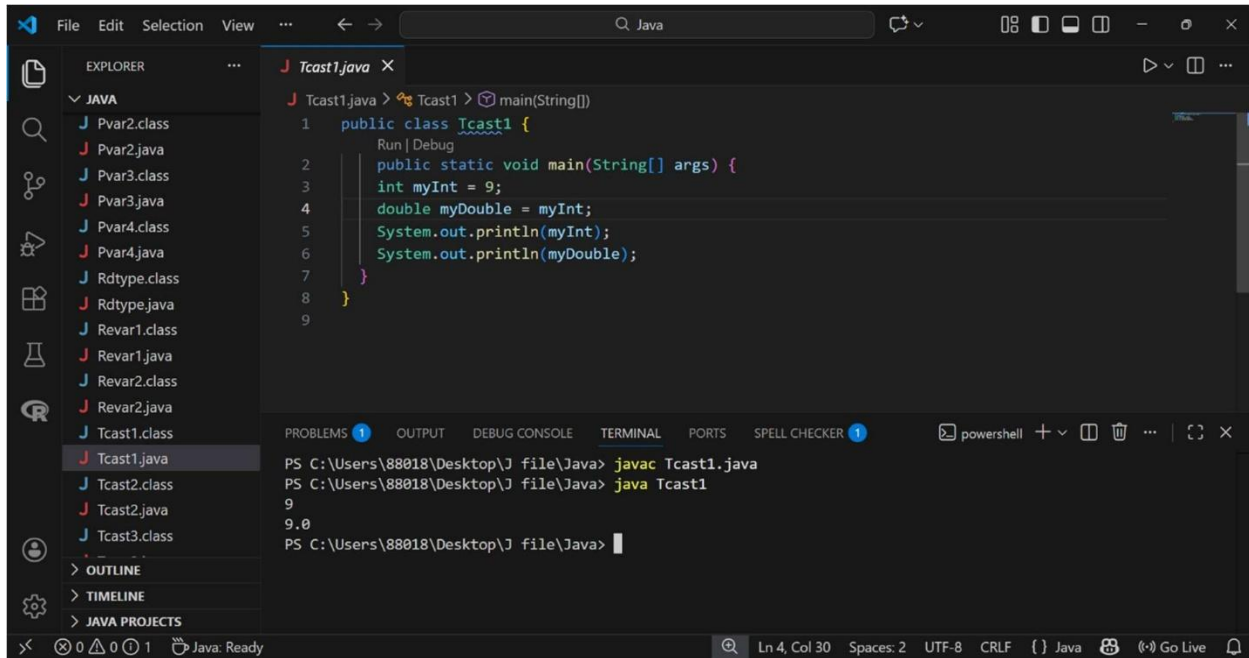
PS C:\Users\88018\Desktop\J file\Java> javac Rdtype.java
PS C:\Users\88018\Desktop\J file\Java> java Rdtype
Number of items: 50
Cost per item: 9.99$
Total cost = 499.5$
PS C:\Users\88018\Desktop\J file\Java>

```

Ln 2, Col 46 Spaces: 2 UTF-8 CRLF () Java Go Live

TypeCasting section:

1. Illustrates Widening Casting, where a smaller type (int) is automatically converted to a larger type (double) by the Java compiler.



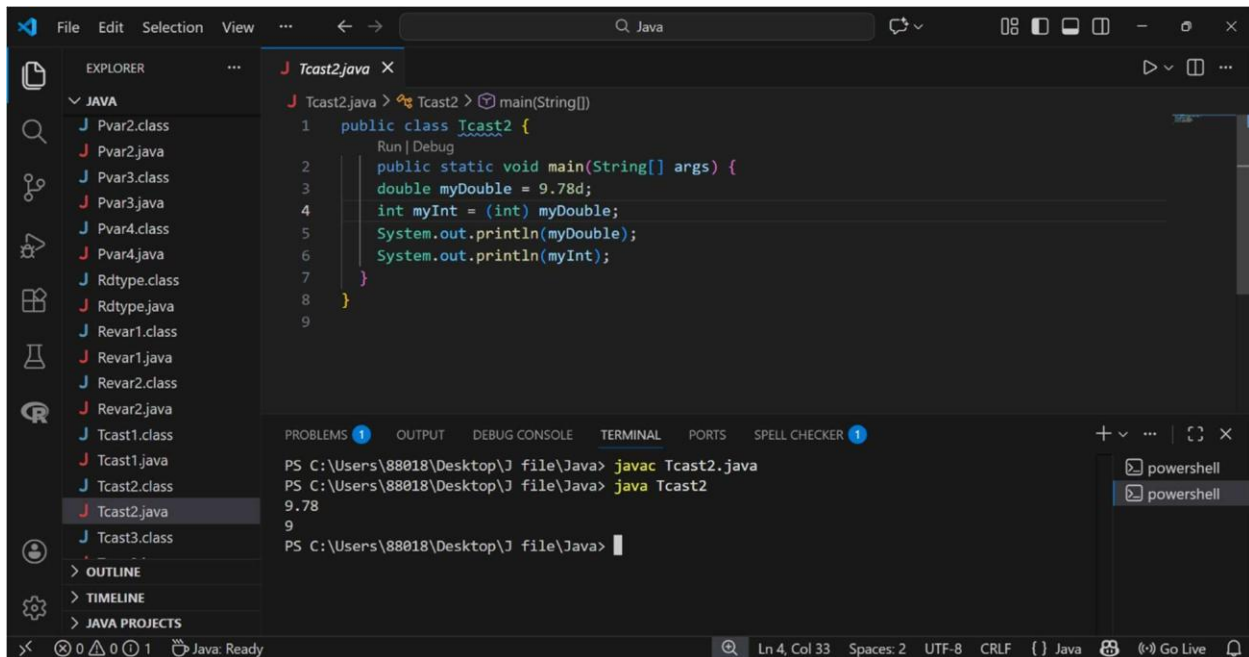
The screenshot shows an IDE with a file explorer on the left containing various Java files. The main editor displays `Tcast1.java` with the following code:

```
1 public class Tcast1 {  
2     public static void main(String[] args) {  
3         int myInt = 9;  
4         double myDouble = myInt;  
5         System.out.println(myInt);  
6         System.out.println(myDouble);  
7     }  
8 }  
9
```

The terminal at the bottom shows the execution of the program:

```
PS C:\Users\88018\Desktop\J file\Java> javac Tcast1.java  
PS C:\Users\88018\Desktop\J file\Java> java Tcast1  
9  
9.0  
PS C:\Users\88018\Desktop\J file\Java>
```

2. Showcases narrowing casting, where a double is manually converted to a smaller int type, resulting in decimal truncation.



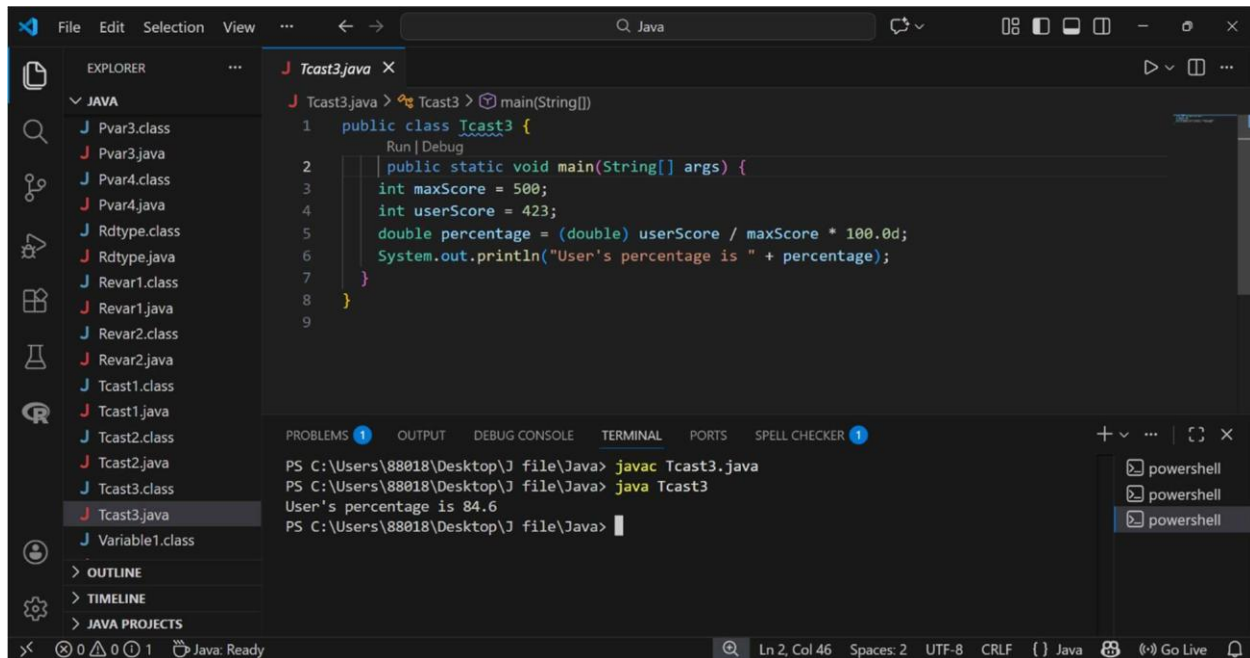
The screenshot shows the same IDE with `Tcast2.java` open. The code is as follows:

```
1 public class Tcast2 {  
2     public static void main(String[] args) {  
3         double myDouble = 9.78d;  
4         int myInt = (int) myDouble;  
5         System.out.println(myDouble);  
6         System.out.println(myInt);  
7     }  
8 }  
9
```

The terminal output shows the result of the narrowing cast:

```
PS C:\Users\88018\Desktop\J file\Java> javac Tcast2.java  
PS C:\Users\88018\Desktop\J file\Java> java Tcast2  
9.78  
9  
PS C:\Users\88018\Desktop\J file\Java>
```

3. Demonstrates a real-world use of type casting to perform floating-point division between two integers to calculate a percentage.



The screenshot shows an IDE with a file explorer on the left containing various Java files. The main editor displays `Tcast3.java` with the following code:

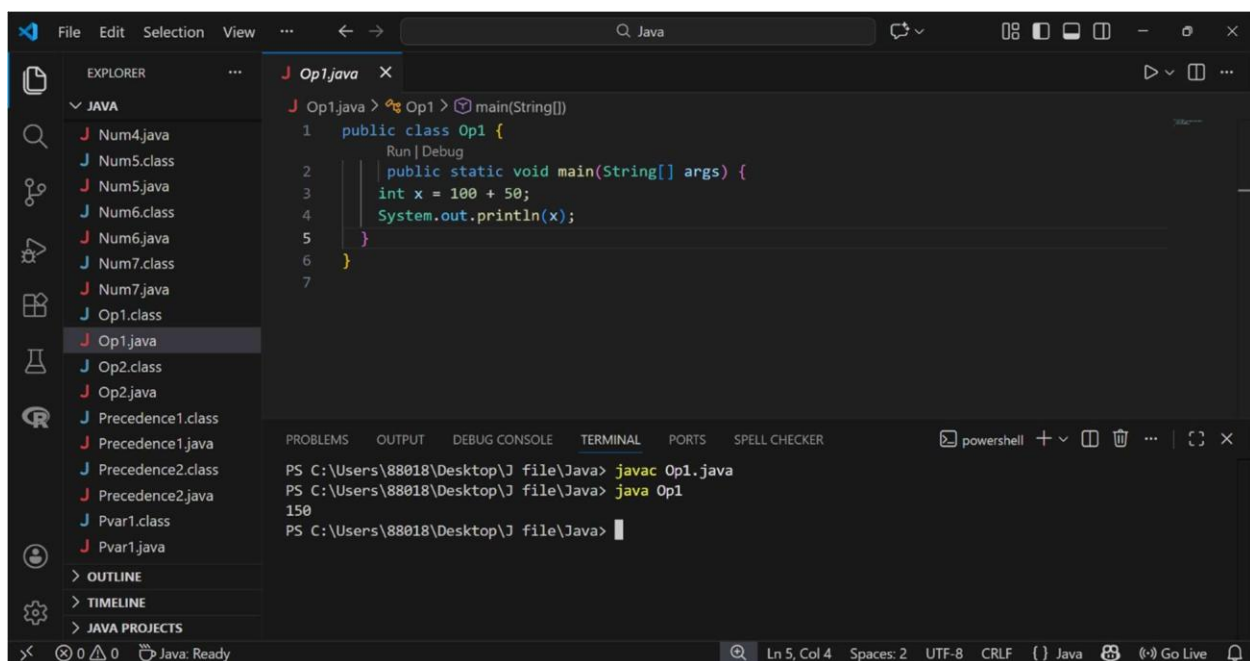
```
1 public class Tcast3 {  
2     public static void main(String[] args) {  
3         int maxScore = 500;  
4         int userScore = 423;  
5         double percentage = (double) userScore / maxScore * 100.0d;  
6         System.out.println("User's percentage is " + percentage);  
7     }  
8 }  
9
```

The bottom panel shows the terminal output:

```
PS C:\Users\88018\Desktop\J file\Java> javac Tcast3.java  
PS C:\Users\88018\Desktop\J file\Java> java Tcast3  
User's percentage is 84.6  
PS C:\Users\88018\Desktop\J file\Java>
```

Operators:

1. The provided Java code demonstrates a basic arithmetic operation where the integer variable `x` is assigned the sum of 100 and 50, and the resulting value, 150, is displayed in the console using the `System.out.println()` method.



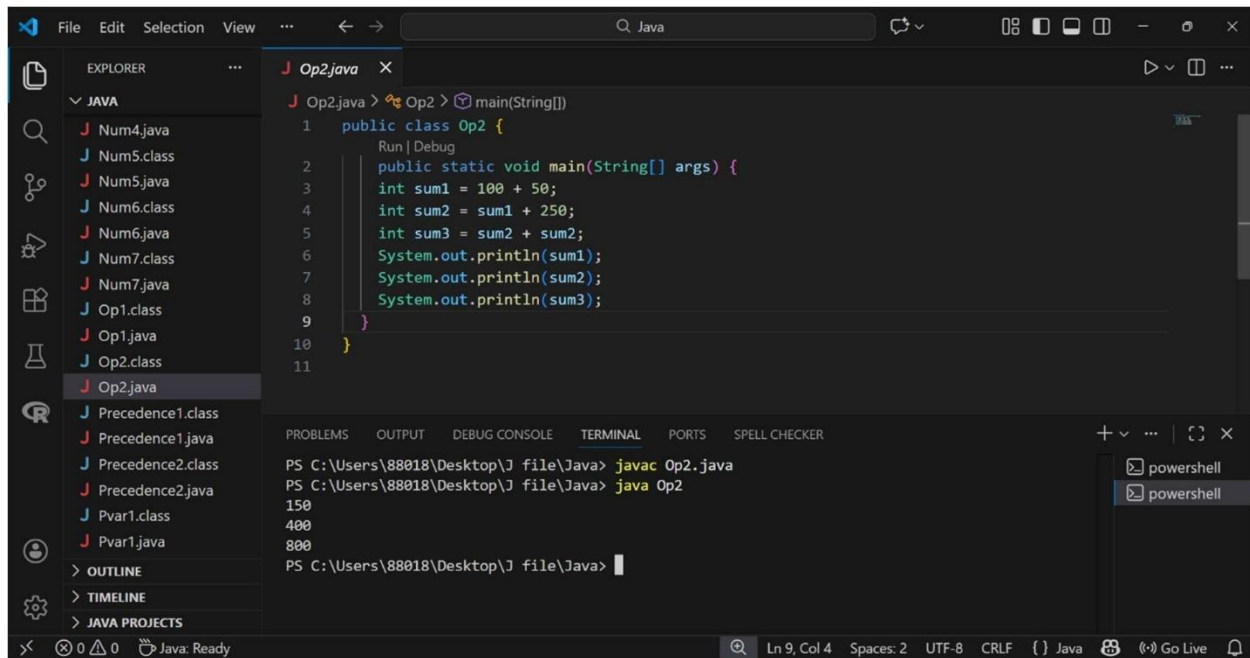
The screenshot shows an IDE with a file explorer on the left containing various Java files. The main editor displays `Op1.java` with the following code:

```
1 public class Op1 {  
2     public static void main(String[] args) {  
3         int x = 100 + 50;  
4         System.out.println(x);  
5     }  
6 }  
7
```

The bottom panel shows the terminal output:

```
PS C:\Users\88018\Desktop\J file\Java> javac Op1.java  
PS C:\Users\88018\Desktop\J file\Java> java Op1  
150  
PS C:\Users\88018\Desktop\J file\Java>
```

2. This Java program illustrates the sequential use of the addition operator to perform calculations using both literal values and previously defined variables, outputting the results of each step to the terminal.



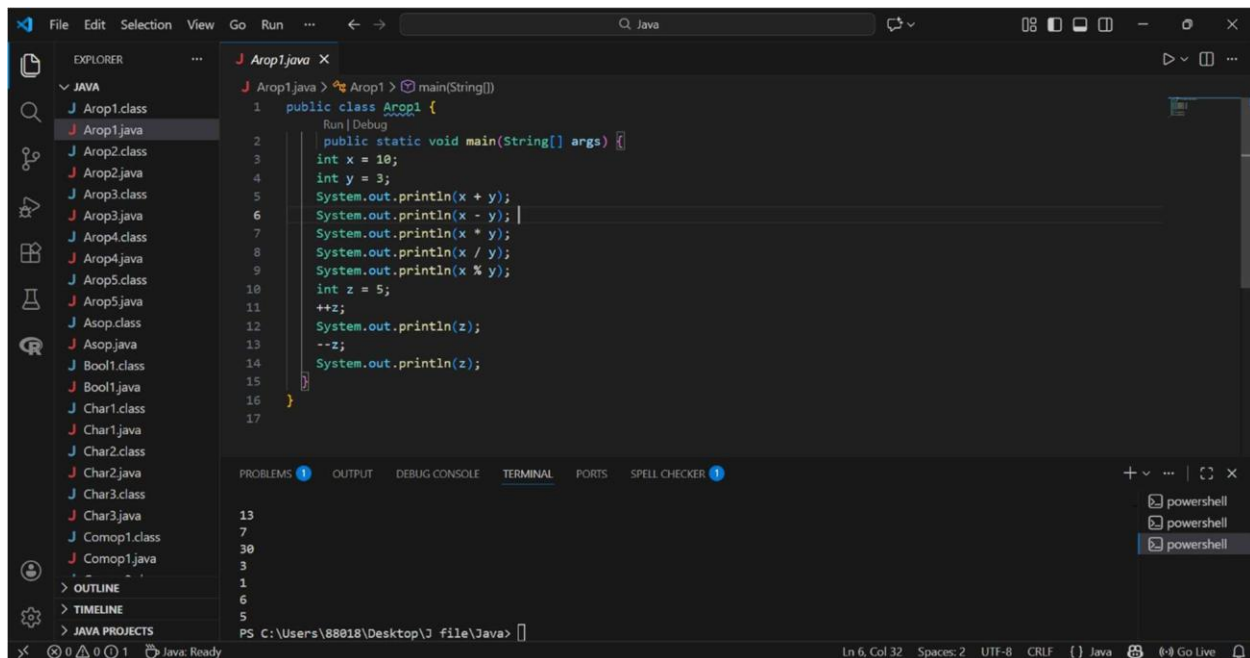
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Num4.java
    Num5.class
    Num5.java
    Num6.class
    Num6.java
    Num7.class
    Num7.java
    Op1.class
    Op1.java
    Op2.class
    Op2.java
    Precedence1.class
    Precedence1.java
    Precedence2.class
    Precedence2.java
    Pvar1.class
    Pvar1.java
  OUTLINE
  TIMELINE
  JAVA PROJECTS

Op2.java
  Op2.java > Op2 > main(String[])
  1 public class Op2 {
  2     public static void main(String[] args) {
  3         int sum1 = 100 + 50;
  4         int sum2 = sum1 + 250;
  5         int sum3 = sum2 + sum2;
  6         System.out.println(sum1);
  7         System.out.println(sum2);
  8         System.out.println(sum3);
  9     }
  10 }
  11

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Op2.java
PS C:\Users\88018\Desktop\J file\Java> java Op2
150
400
800
PS C:\Users\88018\Desktop\J file\Java>

Ln 9, Col 4 Spaces: 2 UTF-8 CRLF () Java Go Live
```

3. This Java program demonstrates the application of basic arithmetic operators (addition, subtraction, multiplication, division, and modulo) alongside prefix increment and decrement operators to manipulate and output integer values.



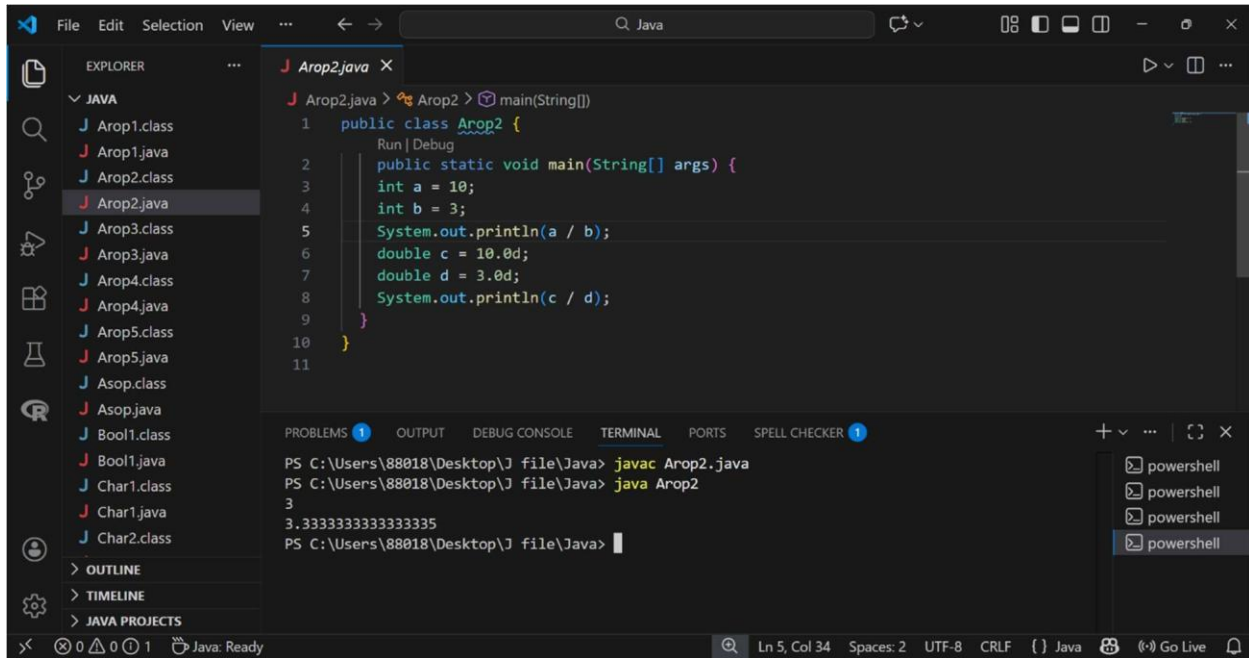
```
File Edit Selection View Go Run ... Java
EXPLORER
  JAVA
    Arop1.class
    Arop1.java
    Arop2.class
    Arop2.java
    Arop3.class
    Arop3.java
    Arop4.class
    Arop4.java
    Arop5.class
    Arop5.java
    Asop.class
    Asop.java
    Bool1.class
    Bool1.java
    Char1.class
    Char1.java
    Char2.class
    Char2.java
    Char3.class
    Char3.java
    Comop1.class
    Comop1.java
  OUTLINE
  TIMELINE
  JAVA PROJECTS

Arop1.java
  Arop1.java > Arop1 > main(String[])
  1 public class Arop1 {
  2     public static void main(String[] args) {
  3         int x = 10;
  4         int y = 3;
  5         System.out.println(x + y);
  6         System.out.println(x - y);
  7         System.out.println(x * y);
  8         System.out.println(x / y);
  9         System.out.println(x % y);
  10        int z = 5;
  11        ++z;
  12        System.out.println(z);
  13        --z;
  14        System.out.println(z);
  15    }
  16 }
  17

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
13
7
30
3
1
6
5
PS C:\Users\88018\Desktop\J file\Java>

Ln 6, Col 32 Spaces: 2 UTF-8 CRLF () Java Go Live
```

4. This Java program highlights the difference between integer and floating-point division by demonstrating how the former truncates decimal values while the double type preserves precision during calculations.



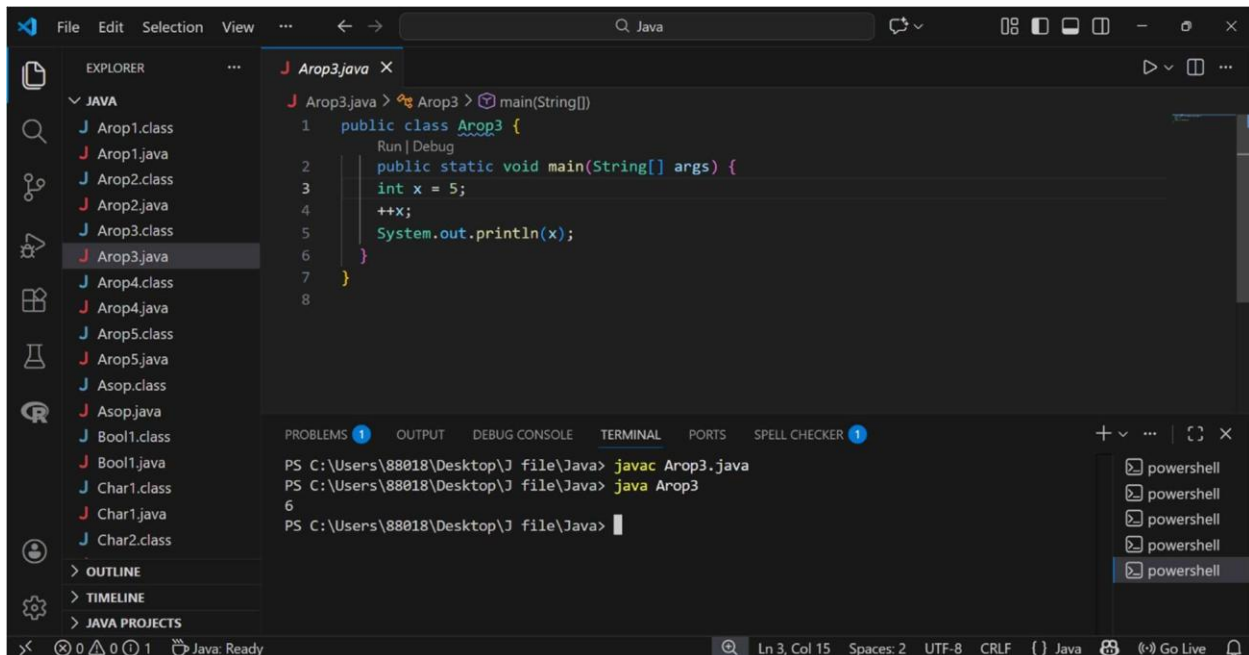
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Arop1.class
    Arop1.java
    Arop2.class
    Arop2.java
    Arop3.class
    Arop3.java
    Arop4.class
    Arop4.java
    Arop5.class
    Arop5.java
    Asop.class
    Asop.java
    Bool1.class
    Bool1.java
    Char1.class
    Char1.java
    Char2.class
  OUTLINE
  TIMELINE
  JAVA PROJECTS

J Arop2.java X
  Arop2.java > Arop2 > main(String[])
  1 public class Arop2 {
    Run | Debug
  2     public static void main(String[] args) {
  3         int a = 10;
  4         int b = 3;
  5         System.out.println(a / b);
  6         double c = 10.0d;
  7         double d = 3.0d;
  8         System.out.println(c / d);
  9     }
 10 }
 11

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Arop2.java
PS C:\Users\88018\Desktop\J file\Java> java Arop2
3
3.3333333333333335
PS C:\Users\88018\Desktop\J file\Java>

Ln 5, Col 34 Spaces: 2 UTF-8 CRLF ( ) Java Go Live
```

5. This Java program demonstrates the use of the prefix increment operator (++x) to increase an integer variable's value by one before it is processed and displayed in the console output.



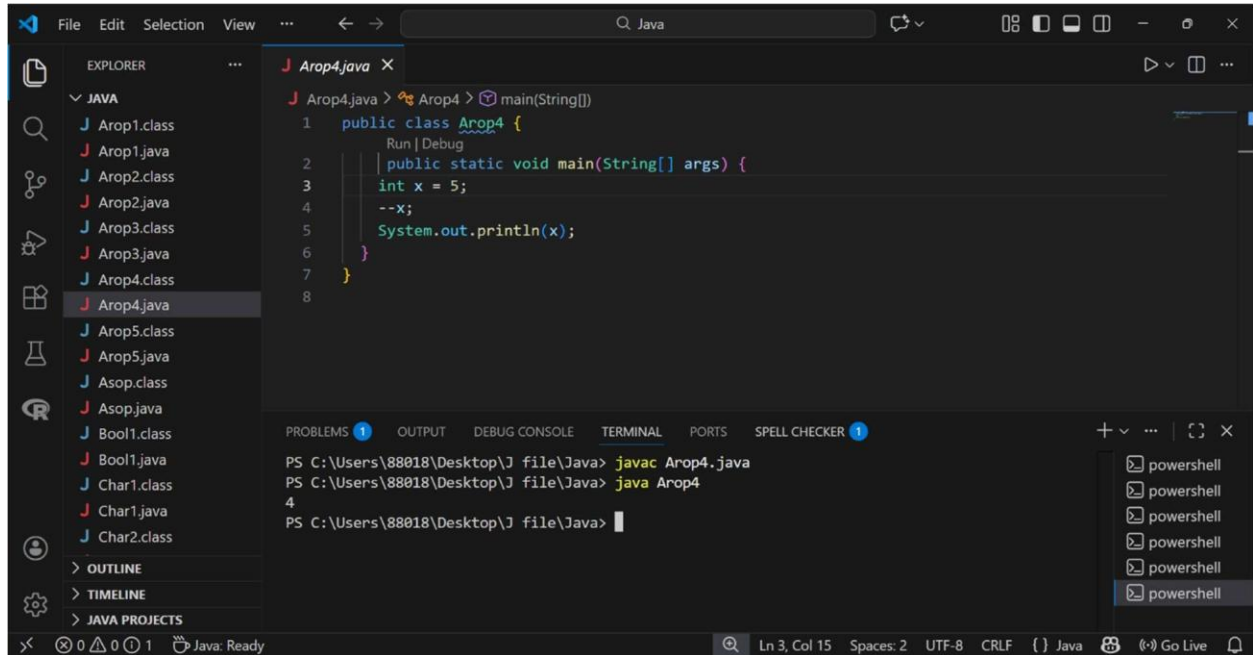
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Arop1.class
    Arop1.java
    Arop2.class
    Arop2.java
    Arop3.class
    Arop3.java
    Arop4.class
    Arop4.java
    Arop5.class
    Arop5.java
    Asop.class
    Asop.java
    Bool1.class
    Bool1.java
    Char1.class
    Char1.java
    Char2.class
  OUTLINE
  TIMELINE
  JAVA PROJECTS

J Arop3.java X
  Arop3.java > Arop3 > main(String[])
  1 public class Arop3 {
    Run | Debug
  2     public static void main(String[] args) {
  3         int x = 5;
  4         ++x;
  5         System.out.println(x);
  6     }
  7 }
  8

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Arop3.java
PS C:\Users\88018\Desktop\J file\Java> java Arop3
6
PS C:\Users\88018\Desktop\J file\Java>

Ln 3, Col 15 Spaces: 2 UTF-8 CRLF ( ) Java Go Live
```

6. This Java program utilizes the prefix decrement operator (`--x`) to decrease the value of an integer variable by one before printing the updated result to the console.



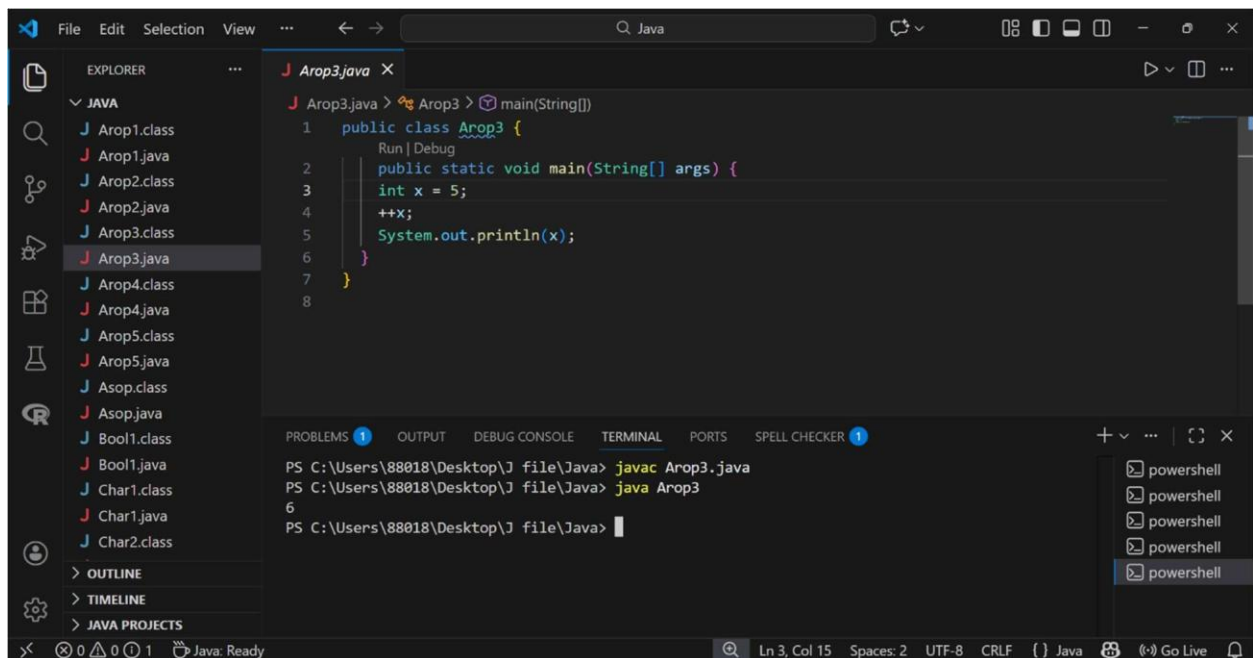
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Arop1.class
    Arop1.java
    Arop2.class
    Arop2.java
    Arop3.class
    Arop3.java
    Arop4.class
    Arop4.java
    Arop5.class
    Arop5.java
    Asop.class
    Asop.java
    Bool1.class
    Bool1.java
    Char1.class
    Char1.java
    Char2.class
  OUTLINE
  TIMELINE
  JAVA PROJECTS

J Arop4.java X
  J Arop4.java > Arop4 > main(String[])
  1 public class Arop4 {
  2     public static void main(String[] args) {
  3         int x = 5;
  4         --x;
  5         System.out.println(x);
  6     }
  7 }
  8

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Arop4.java
PS C:\Users\88018\Desktop\J file\Java> java Arop4
4
PS C:\Users\88018\Desktop\J file\Java>

powershell
powershell
powershell
powershell
powershell
```

7. This Java program demonstrates the use of postfix increment (`++`) and postfix decrement (`--`) operators to iteratively modify a variable's value, simulating a simple counter for tracking items or individuals.



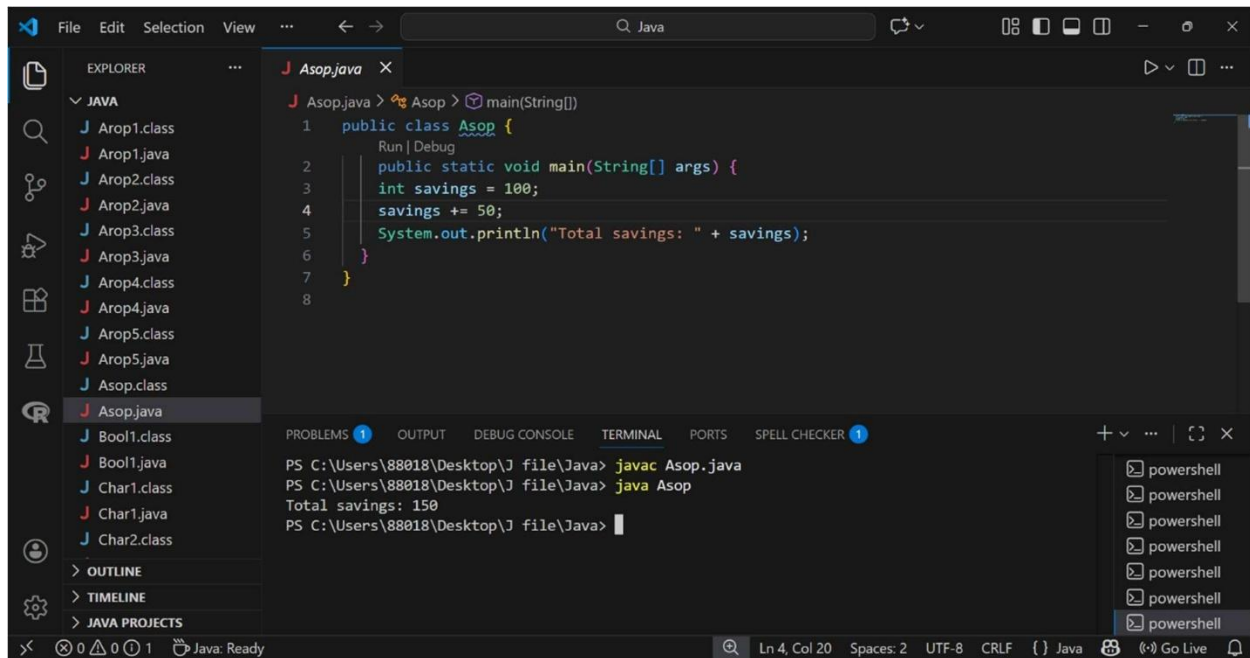
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Arop1.class
    Arop1.java
    Arop2.class
    Arop2.java
    Arop3.class
    Arop3.java
    Arop4.class
    Arop4.java
    Arop5.class
    Arop5.java
    Asop.class
    Asop.java
    Bool1.class
    Bool1.java
    Char1.class
    Char1.java
    Char2.class
  OUTLINE
  TIMELINE
  JAVA PROJECTS

J Arop3.java X
  J Arop3.java > Arop3 > main(String[])
  1 public class Arop3 {
  2     public static void main(String[] args) {
  3         int x = 5;
  4         ++x;
  5         System.out.println(x);
  6     }
  7 }
  8

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Arop3.java
PS C:\Users\88018\Desktop\J file\Java> java Arop3
6
PS C:\Users\88018\Desktop\J file\Java>

powershell
powershell
powershell
powershell
powershell
```


8. This Java program demonstrates the use of the add on assignment operator (+=) to update a variable's value by adding a specific amount to its existing total and displaying the result.



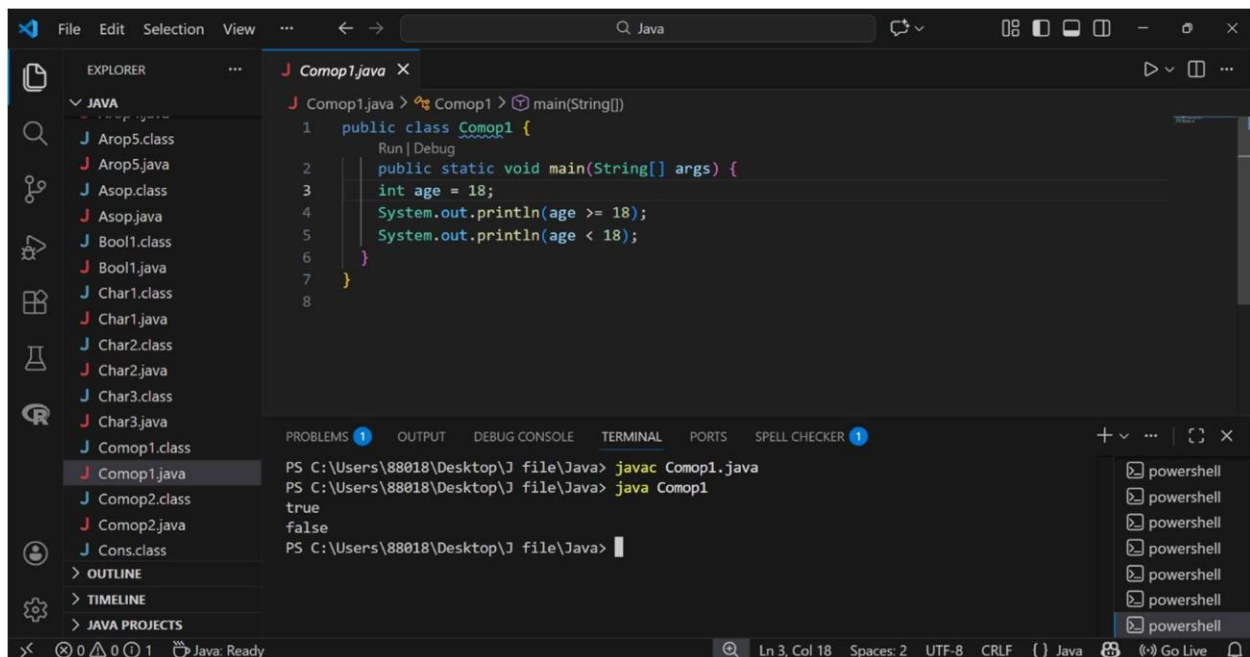
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    J Arop1.class
    J Arop1.java
    J Arop2.class
    J Arop2.java
    J Arop3.class
    J Arop3.java
    J Arop4.class
    J Arop4.java
    J Arop5.class
    J Arop5.java
    J Asop.class
    J Asop.java
    J Bool1.class
    J Bool1.java
    J Char1.class
    J Char1.java
    J Char2.class
  > OUTLINE
  > TIMELINE
  > JAVA PROJECTS

J Asop.java X
J Asop.java > % Asop > main(String[])
1 public class Asop {
  Run | Debug
  public static void main(String[] args) {
2   int savings = 100;
3   savings += 50;
4   System.out.println("Total savings: " + savings);
5 }
6
7
8

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Asop.java
PS C:\Users\88018\Desktop\J file\Java> java Asop
Total savings: 150
PS C:\Users\88018\Desktop\J file\Java>

powershell
powershell
powershell
powershell
powershell
powershell
```

9. This Java program demonstrates the use of comparison operators, specifically "greater than or equal to" (>=) and "less than" (<), to evaluate a numerical condition and return a boolean true or false result.



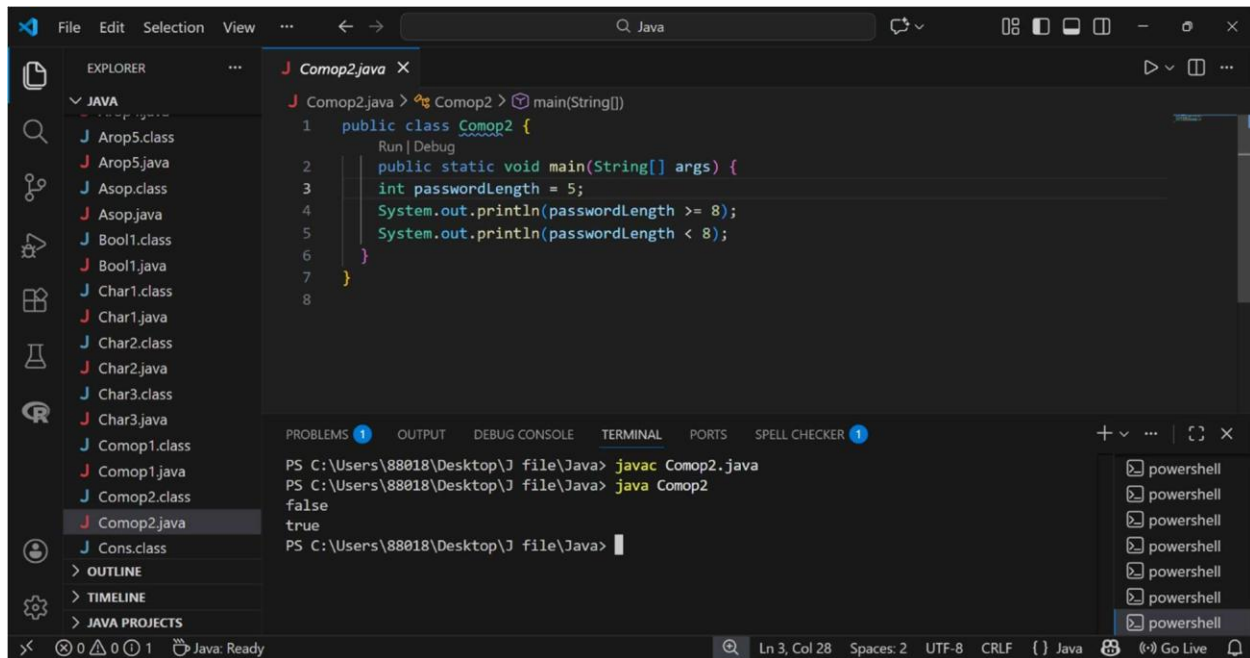
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    J Arop5.class
    J Arop5.java
    J Asop.class
    J Asop.java
    J Bool1.class
    J Bool1.java
    J Char1.class
    J Char1.java
    J Char2.class
    J Char2.java
    J Char3.class
    J Char3.java
    J Comop1.class
    J Comop1.java
    J Comop2.class
    J Comop2.java
    J Cons.class
  > OUTLINE
  > TIMELINE
  > JAVA PROJECTS

J Comop1.java X
J Comop1.java > % Comop1 > main(String[])
1 public class Comop1 {
  Run | Debug
  public static void main(String[] args) {
2   int age = 18;
3   System.out.println(age >= 18);
4   System.out.println(age < 18);
5 }
6
7
8

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Comop1.java
PS C:\Users\88018\Desktop\J file\Java> java Comop1
true
false
PS C:\Users\88018\Desktop\J file\Java>

powershell
powershell
powershell
powershell
powershell
powershell
```


10. This Java program utilizes comparison operators ($\geq$ and $\leq$) to evaluate if a variable representing password length meets a minimum character requirement, outputting the boolean results to the console.



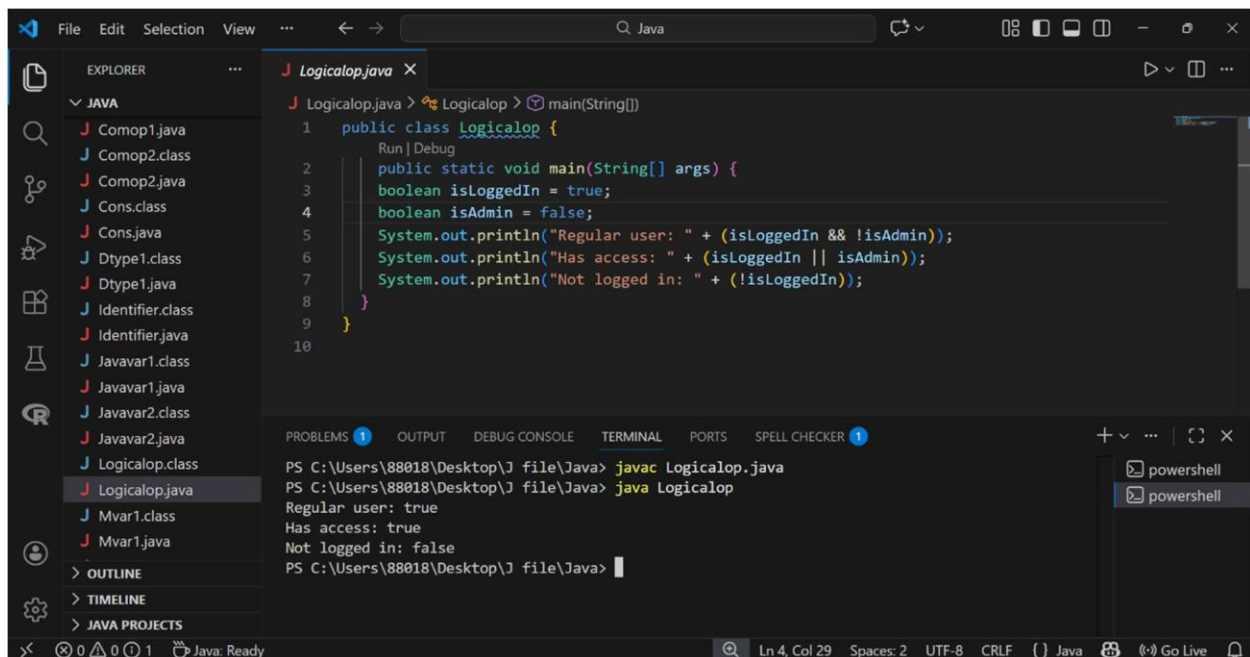
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Comop2.java
    Comop1.class
    Comop2.class
    Cons.class
    Cons.java
    Dtype1.class
    Dtype1.java
    Identifier.class
    Identifier.java
    Javavar1.class
    Javavar1.java
    Javavar2.class
    Javavar2.java
    Logicalop.class
    Logicalop.java
    Mvar1.class
    Mvar1.java
    Comop1.java
    Comop2.java
    Comop5.java
    Asop.class
    Asop.java
    Bool1.class
    Bool1.java
    Char1.class
    Char1.java
    Char2.class
    Char2.java
    Char3.class
    Char3.java

J Comop2.java X
  J Comop2.java > Comop2 > main(String[])
  1 public class Comop2 {
    Run | Debug
  2     public static void main(String[] args) {
  3         int passwordLength = 5;
  4         System.out.println(passwordLength >= 8);
  5         System.out.println(passwordLength < 8);
  6     }
  7 }
  8

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Comop2.java
PS C:\Users\88018\Desktop\J file\Java> java Comop2
false
true
PS C:\Users\88018\Desktop\J file\Java>

Ln 3, Col 28 Spaces: 2 UTF-8 CRLF () Java Go Live
```

11. This Java program demonstrates the application of logical operators, specifically AND (&&), OR (||), and NOT (!), to evaluate boolean variables and determine conditional outcomes based on user login and administrator status.



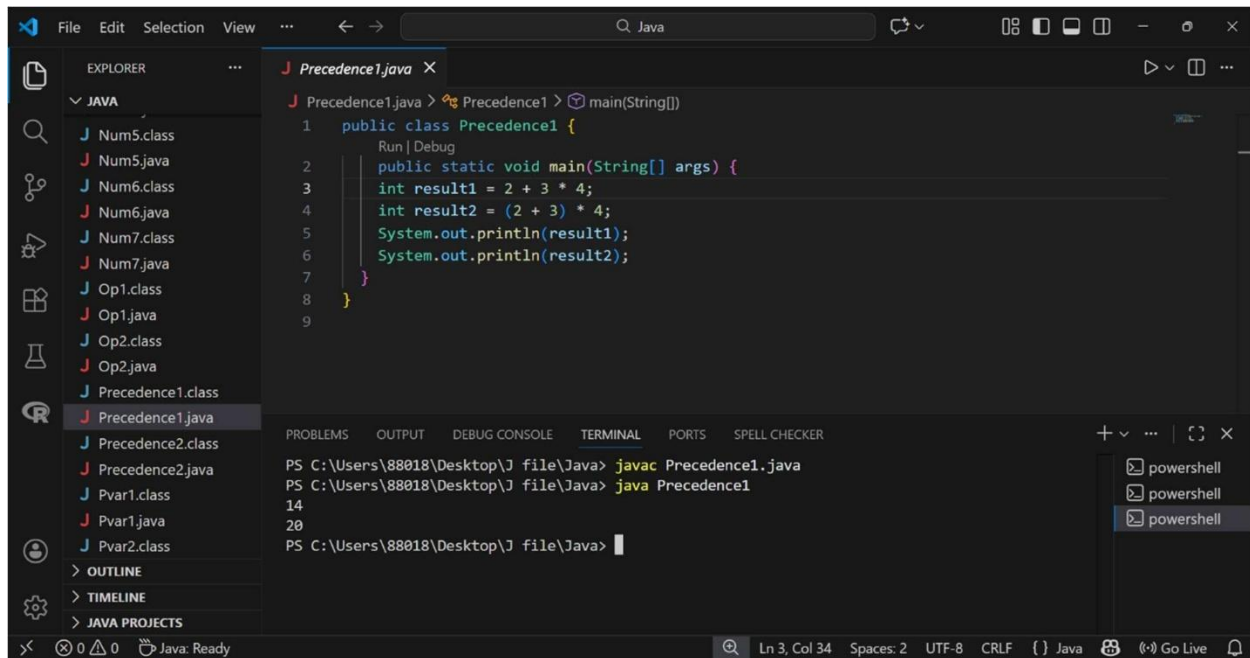
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Comop1.java
    Comop2.class
    Comop2.java
    Cons.class
    Cons.java
    Dtype1.class
    Dtype1.java
    Identifier.class
    Identifier.java
    Javavar1.class
    Javavar1.java
    Javavar2.class
    Javavar2.java
    Logicalop.class
    Logicalop.java
    Mvar1.class
    Mvar1.java
    Comop1.java
    Comop2.java
    Comop5.java
    Asop.class
    Asop.java
    Bool1.class
    Bool1.java
    Char1.class
    Char1.java
    Char2.class
    Char2.java
    Char3.class
    Char3.java

J Logicalop.java X
  J Logicalop.java > Logicalop > main(String[])
  1 public class Logicalop {
    Run | Debug
  2     public static void main(String[] args) {
  3         boolean isLoggedIn = true;
  4         boolean isAdmin = false;
  5         System.out.println("Regular user: " + (isLoggedIn && !isAdmin));
  6         System.out.println("Has access: " + (isLoggedIn || isAdmin));
  7         System.out.println("Not logged in: " + (!isLoggedIn));
  8     }
  9 }
  10

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 1
PS C:\Users\88018\Desktop\J file\Java> javac Logicalop.java
PS C:\Users\88018\Desktop\J file\Java> java Logicalop
Regular user: true
Has access: true
Not logged in: false
PS C:\Users\88018\Desktop\J file\Java>

Ln 4, Col 29 Spaces: 2 UTF-8 CRLF () Java Go Live
```

12. This Java program illustrates operator precedence by showing how multiplication is prioritized over addition by default, and how parentheses can be used to override this order to perform addition first.



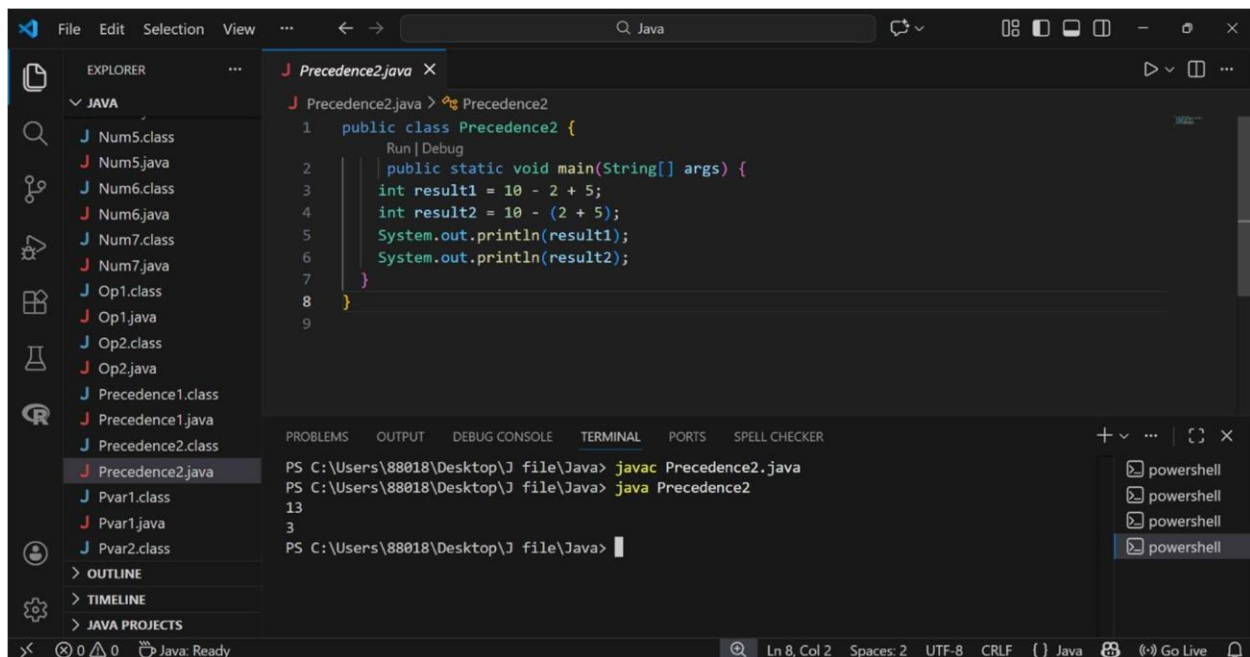
```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Num5.class
    Num5.java
    Num6.class
    Num6.java
    Num7.class
    Num7.java
    Op1.class
    Op1.java
    Op2.class
    Op2.java
    Precedence1.class
    Precedence1.java
    Precedence2.class
    Precedence2.java
    Pvar1.class
    Pvar1.java
    Pvar2.class
  OUTLINE
  TIMELINE
  JAVA PROJECTS

J Precedence1.java X
J Precedence1.java > Precedence1 > main(String[])
1 public class Precedence1 {
  Run | Debug
2   public static void main(String[] args) {
3     int result1 = 2 + 3 * 4;
4     int result2 = (2 + 3) * 4;
5     System.out.println(result1);
6     System.out.println(result2);
7   }
8 }
9

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Precedence1.java
PS C:\Users\88018\Desktop\J file\Java> java Precedence1
14
20
PS C:\Users\88018\Desktop\J file\Java>

Ln 3, Col 34 Spaces: 2 UTF-8 CRLF () Java Go Live
```

13. This Java program demonstrates operator associativity and precedence by showing how subtraction and addition are evaluated from left to right by default, and how parentheses can be used to group operations to change the final result.



```
File Edit Selection View ... Java
EXPLORER
  JAVA
    Num5.class
    Num5.java
    Num6.class
    Num6.java
    Num7.class
    Num7.java
    Op1.class
    Op1.java
    Op2.class
    Op2.java
    Precedence1.class
    Precedence1.java
    Precedence2.class
    Precedence2.java
    Pvar1.class
    Pvar1.java
    Pvar2.class
  OUTLINE
  TIMELINE
  JAVA PROJECTS

J Precedence2.java X
J Precedence2.java > Precedence2
1 public class Precedence2 {
  Run | Debug
2   public static void main(String[] args) {
3     int result1 = 10 - 2 + 5;
4     int result2 = 10 - (2 + 5);
5     System.out.println(result1);
6     System.out.println(result2);
7   }
8 }
9

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
PS C:\Users\88018\Desktop\J file\Java> javac Precedence2.java
PS C:\Users\88018\Desktop\J file\Java> java Precedence2
13
3
PS C:\Users\88018\Desktop\J file\Java>

Ln 8, Col 2 Spaces: 2 UTF-8 CRLF () Java Go Live
```

